PROJECT A.M.P.E.R.E.

Automated Monitoring of Parasitic Electrical Responses and Events

Anish Shriram
Electrical Engineering Intern
Legrand North and Central America – The Wiremold Company
August 2026

Notice on Use of Artificial Intelligence:

Microsoft's Copilot Prometheus 2.0 (Utilizing OpenAI's GPT 5.6) *was used for assistance during planning and debugging sessions, specifically in interpreting Linux 'Raspbian' command line best practices. It was also used to generate a 'preflight checklist' and an error log, so that test iterations could be booted up and errors could be resolved quicker.*

Anthropic's Claude Opus 5.0 *was used for assistance in writing and simulating hardware abstraction layer tests as well as debugging camera and oscilloscope communication pathways. It was also used for assistance in building and debugging the CLI, lifecycle, and monitoring service, and to correct legacy documentation and minor errors in reporting. It was also used for assistance in building analysis methods B and C.*

Microsoft's Github Copilot (Utilizing Microsoft Build's MAI-1.1-Code-FLASH) *was used for assistance in building forced_diagnostic_analysis.py, a currently offline file used to analyze waveforms taken for diagnostic purposes. It was also used for assistance in refining the trip analysis algorithm, and in refining endpoint documentation from algorithm version to version.*

Microsoft's Github Copilot (Utilizing Anthropic's Claude Sonnet 5.0) *was used for assistance in building the sequencer, state machine, persistence and recovery service, and charging gate classifier. It was also used for assistance in debugging the service suite at large.*

This codebase, and the hardware it interacts with, was architected, directed, and thoroughly reviewed by me, Anish Shriram, and my superiors at Legrand North and Central America. AI generation was utilized strictly as an execution tool to implement the specific system designs and instructions provided by me. I maintain full ownership, responsibility, and verification of the final quality of this project.

I would like to thank Dr. S.M. Rakiul Islam as well as Mr. Jeffrey Hemingway for their assistance and insights on engineering best practices when putting this project together.

SYSTEM DESIGN AND VALIDATON REPORT

Executive Summary

Charge Circuit Interrupting Devices (CCIDs) are the ground fault protective element inside electric vehicle supply equipment (EVSE). A CCID must clear a leakage fault within a bounded time, and confidence in that behavior requires exercising a device across thousands of repeated fault cycles rather than trusting a single measurement.

Project A.M.P.E.R.E. is a Raspberry Pi driven test rig that injects a controlled ground fault into a live EVSE, captures the resulting transient on a digital oscilloscope, verifies via machine vision that the fault was injected only while the EVSE was genuinely charging, and records every cycle's raw waveform and verdict to a crash-safe, resumable data store for extended campaigns.

This report specifies the system's requirements, electrical and software architecture, measurement methodology (including the specific algorithms used to convert a raw waveform into a trip-time verdict), and the commissioning process that validated the rig before it was trusted to run unattended.

1. Introduction

When a leakage fault occurs, the charge circuit interrupting device must interrupt the circuit within a bounded time. UL 2231-2 is the governing standard, and it requires exercising the device across many repeated fault events and confirming its response stays consistently within bounds without an occasional slow response hiding in the tail of the distribution. Each cycle requires safely energizing the rig, injecting a fault at a known point in time, capturing a fast transient with the right trigger and timebase settings, reading the result, and safely de-energizing again. A human operator can do this a handful of times per session, and performing the required 6,000 cycle campaign is wasteful of engineering resources. That gap is why this system exists.

Scope: This document will define what the system must do (Section 3), the physical, instrumentation, and software architecture (Section 4), the operating sequence (Section 5), the measurement methodology (Section 6), the data architecture and crash safety (Section 7), automatic failure recovery (Section 8), verification and pre hardware validation ladder (Section 9), and known open limitations (Section 10).

Out of Scope: This document does not cover the execution or results of any specific test campaign. The 6,000 cycle endurance campaign run on this system is documented separately in Addendum I. It does not constitute a UL 2231-2 certification record, in fact the measurement endpoint used by the system is, as of this writing, the Wiremold Company’s own provisional interpretation of the standard, pending confirmation against the published standard text.

2. System Requirements

ID	Requirement
SAF-01	Every contactor's default state shall be open, independent of any software having run.
SAF-02	K3 shall close only once K1 and K2 are both already commanded closed, and only against a single use / per cycle charging gate authorization token.
SAF-03	K1 and K2 shall not be commanded open while K3 is commanded closed.
SAF-04	K3's closed duration within a single cycle shall never exceed a fixed hard limit (300ms), independent of whether the oscilloscope reports a successful acquisition.
SAF-05	A full de-energization attempt shall be made from every halt, completion, and unhandled exception path, and shall attempt every step even if an earlier step fails.
MEAS-01	Trip time shall be resolved as an onset instant and end instant read from the waveform's own capture samples and timebase; the oscilloscope's hardware trigger event shall never be assumed to mark the fault onset.
MEAS-02	Every raw waveform capture shall be preserved in full.
MEAS-03	The trip time algorithm shall be a versioned boundary, and any defect is corrected by adding a new algorithm version.
MEAS-04	Verdict classification (PASS/FAIL/NO_TRIP) shall be decided only from the computed trip time against the two configured limits; waveform sanity checks shall be recorded but shall never veto a verdict, except checks that represent a distinct actionable safety condition (K3 pretrigger current, record too short to trust a no-trip conclusion).
MEAS-05	The measurement endpoint definition shall be explicitly stated and frozen into the run configuration.

VIS-01	A fault shall be injected only while the EVSE has been independently confirmed, via machine vision, to be in a genuine charging state.
VIS-02	The vision subsystem shall never be capable of halting the campaign on its own failure and shall never compute or influence a trip time measurement,
OPS-01	The system shall automatically recover from a bounded number of consecutive non-DUT halts (5) before requiring a human, and a tighter bound (3) for a NO_TRIP verdict.
OPS-02	A halted run shall remain halted across process restarts until an operator explicitly overrides the halt.
OPS-03	Camera and oscilloscope connections shall be periodically and reactively refreshed to mitigate gradual USB/driver degradation.
DAT-01	A cycle shall be considered committed only once its artifacts, CSV row, and run state update have all been durably written in that order.
DAT-02	The exact configuration in force for a run shall be frozen into a canonical hash at run start; a resume against a silently different configuration shall be refused.
DAT-03	Every stored measurement shall be traceable from the CSV row alone, to the exact algorithm version and configuration that produced it.

3. System Architecture

3.1 Electrical Architecture and Safety Interlocks

The rig centers on a JoinTech Gen 2 EV Charger (Model JNT-EVM003/48AC/01C/SR/RF/WF/4G), driven by three independently controlled single-pole contactors. Contactors K1 and K2 switch the L1 and L2 mains legs feeding the EVSE, and Contactor K3 switches a separate leakage injection path that introduces the ground fault current the CCID is expected to detect and clear. Each is driven by its own ZX-517 dual-MOSFET opto-driver board from its own isolated 12 VDC supply, commanded independently by a Raspberry Pi

GPIO line (K1 on GPIO17, K2 on GPIO27, K3 on GPIO22). Because the EVSE needs to believe a vehicle is present and requesting a charge before it will energize, the rig uses a static control pilot spoof device rather than a physical vehicle.

On the software side, every GPIO output initializes inactive and drives active high, so the rig's default state is de-energized whether or not software has run yet. A contactor is treated as open unless something has actively and recently commanded it closed, and the system has no auxiliary contact or voltage feedback confirming a commanded state was physically achieved (this gap addressed in Section 10). The layered defaults, single use gate token, and independent software backstop reduce the likelihood and duration of unintended energization but they do not constitute independent confirmation of physical contactor state.

The leakage injection path carries a 300 ms hard backstop on how long K3 may remain closed within a cycle (SAF-04), enforced by polling elapsed time in short slices rather than a single blocking wait, so a slow or stalled scope acquisition can never hold K3 closed past its deadline. Closing K3 at all is gated on a single-use, per-cycle authorization token issued only once the charging-gate vision check (Section 6) has confirmed the EVSE is genuinely charging (SAF-02). A fault is never injected into a rig that hasn't been independently confirmed to be in the state the test assumes.

3.2 Instrumentation

Trip time measurement uses a Keysight MSO-X 2014A digital oscilloscope, connected over USB and driven entirely through a Python VISA stack (PyVISA, PyVISA-py, PyUSB), since no ARM build of a vendor VISA runtime exists for this platform. The scope is configured for a single, edge triggered acquisition per cycle: channel 1 through a 10x, 300 V CAT II passive probe, positive-edge trigger at +20 V with DC trigger coupling and noise-reject disabled, and a full record at 50 ms/div with a centered timebase. This timebase is deliberately generous relative to the pass/fail limits evaluated, so a record is always long enough to either observe the current collapse directly or conclusively rule out a trip within the allowed window. `:WAVeform:POINts:MODE RAW` plus `POINts MAXimum` is required explicitly as the alternative `NORMal` mode returns roughly 1,000 points regardless of the true 1 Mpt memory depth, which would silently truncate every capture.

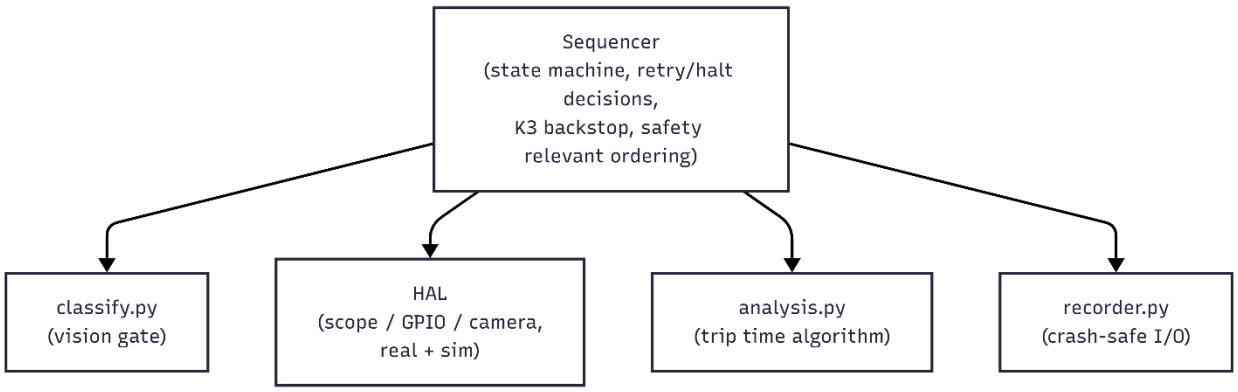
Charging state confirmation uses a Logitech C270 USB webcam aimed at the EVSE's own status LED, read through a calibrated region of interest. Exposure is fixed and manually locked (auto exposure and auto white balance both proved unreliable for consistent color classification and are actively worked around). The camera's device path is resolved through a udev rule keyed to its USB serial number (`deploy/99-c270-camera.rules` to `/dev/ccid_camera`) rather

than a raw `/dev/videoN` index, since a USB re-enumeration event during initial testing demonstrated that a numeric index is not stable.

All contactor switching goes through the Raspberry Pi's native GPIO via the `gpiozero` library on its `lgpio` backend.

3.3 Control and Software Architecture

Control software is organized around a single per-cycle state machine (`ccid/sequencer.py`) that walks a cycle through a fixed sequence of states from a safe, de-energized starting state, through mains closing and the vision charging-gate wait, through scope configuration and arming, through leakage injection and acquisition, back to a fully de-energized state. This cycles regardless of whether the cycle produces a pass, a fail, or a halt. Every hardware subsystem the sequencer depends on sits behind a hardware abstraction layer (HAL) with both a real driver and a simulated driver reproducing the same interface and the same class of failure behavior in software only.



Which mode each subsystem runs in (real or simulated) is an independent setting in `config.yaml`. This is what allows the software itself to be exercised by a deterministic automated test suite (364 tests) that runs in seconds and touches no real hardware, and what allowed individual subsystems to be switched from simulated to real one at a time during commissioning rather than all at once.

Configuration is loaded from a single YAML file, validated strictly against an explicit set of recognized keys (an unrecognized key fails at load time), and reduced to a canonical hash frozen into every run's own record at the moment that run starts (DAT-02, detailed in Section 7).

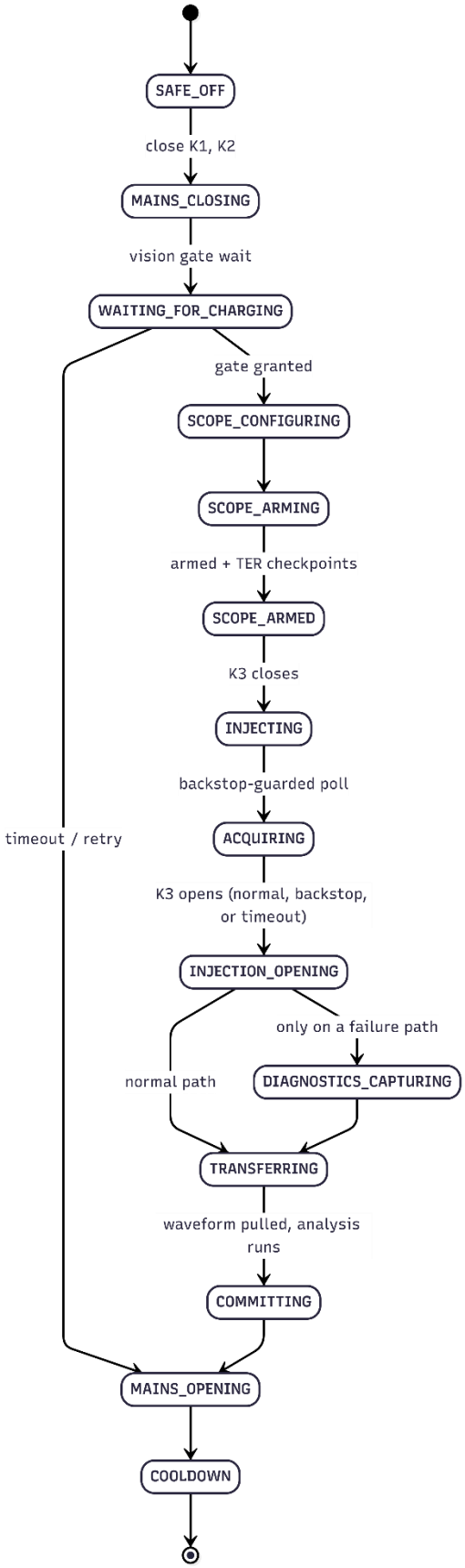
3.4 Safety Functions

- The K3 Interlock (SAF-02, SAF-03) is enforced independently in both `gpio_real.py` and `gpio_sim.py`
- The K3 300 ms backstop (SAF-04) is sequencer level (`Sequencer._poll_acquisition_with_backstop`) because K3 must open by a deadline regardless of what the scope reports.
- `safe_off`'s strict K3 to K2 to K1 open ordering and 'full attempt even on partial failure' semantics (SAF-05) live in `ccid/safety.py`, called from every halt/retry/competion path in the sequencer. It lives again, redundantly, in `ccid.main._execute_campaign`'s own `finally` block as a second independent backstop.
- Vision can never halt or kill the campaign on its own failure (VIS-02) functions by letting a camera fault degrade to a fixed wait and thus the run can continue in a flagged degraded mode, using a 60s wait instead of a real time classifier).
- Outbound monitoring can never halt the campaign.
- A crash mid cycle can never lose or double count a cycle (DAT-01) is guaranteed by the commit order: artifacts to CSV to runstate to heartbeat, plu atomic `runstate.json` replacement plus orphan reconciliation on resume together.
- The sequencer persists the full type/message/traceback/cycle state to a diagnostics artifact before the halt path collapses it down to just the exception's class name so that an unexpected exception does not disappear without a trace.

4. Operating Sequence

Every hardware subsystem the sequencer depends on is described in detail, including the K3 backstop/forced-diagnostic poll loop and every branch off this path (retry, degrade, halt), in the codebase's own reference documentation (`docs/sequencer-and-state-machine.md`).

This report summarizes only what is needed to understand the measurement and safety story.


SAFE_OFF
close K1, K2
MAINS_CLOSING
vision gate wait
WAITING_FOR_CHARGING
gate granted
SCOPE_CONFIGURING
SCOPE_ARMING
armed + TER checkpoints
SCOPE_ARMED
K3 closes
INJECTING
backstop-guarded poll
ACQUIRING
timeout / retry
K3 opens (normal, backstop, or timeout)
INJECTION_OPENING
only on a failure path
normal path
DIAGNOSTICS_CAPTURING
TRANSFERRING
waveform pulled, analysis runs
COMMITTING
MAINS_OPENING
COOLDOWN

5. Measurement Method

5.1 Signal Definition

Trip time is defined as the elapsed time between fault onset and fault current collapse:

$$t_{\text{trip}} = t_{\text{end}} - t_0$$

Neither t_0 or t_{end} is available for free from the instrumentation. The oscilloscope's hardware trigger is deliberately never assumed to mark t_0 (MEAS-01), because the trigger comparator can fire up to roughly half a mains cycle after the fault current actually begins to flow. Treating the trigger as t_0 would systematically overstate every trip time by an amount that depends on line phase at the moment of injection, not the device under test. t_0 is resolved from the cleanest available source in a fixed order of preference:

1. An explicit per-cycle sidecar timestamp if supplied (not currently available)
2. A timestamp embedded in the scope's own preamble if populated (not currently available)
3. An onset detected directly from the shape of the captured waveform itself (used)

The captured signal is rectified before any envelope processing. For a sampled voltage $\mathrm{v}[n]$,

$$\mathrm{x}[n] = |v[n]|$$

The leakage current is AC and crosses zero twice per mains cycle while still flowing. A naive threshold/width detector on the raw signal measures one half cycle (~8.33 ms at 60 Hz) instead of the actual bust duration. Every detection step in this system therefore operates on an envelope of $\mathrm{x}[n]$ rather than on a raw threshold crossing.

The envelope window, in samples, is:

$$N_w = \text{round}\left(C_w \cdot \frac{f_s}{f_{\text{line}}}\right)$$

with C_w = 0.5 and f_{line} = 60 Hz. This gives a fixed window duration of $\frac{C_w}{f_{\text{line}}} = 8.333$ ms. The exact sample count N_w depends on the per-capture sample interval $\mathrm{dt} = \frac{1}{f_s}$, which is read from the oscilloscope's own preamble (`x_increment`) at analysis time rather than fixed in `config.yaml`. The scope's actual acquisition is instrument reported metadata, not a system configured constant.

The specific placement of t_0 and t_{end} within a waveform is this system's own working interpretation of UL 2231-2, frozen into `config.yaml` so it cannot drift silently between cycles or campaigns. That definition remains provisional (MEAS-05). It has not, as of this writing, been confirmed against the published standard text, and the two numeric limits evaluated against (a pass limit of 24.97 ms and a no trip limit of 100 ms) should be read as the thresholds this system currently applies, not as an independently certified interpretation.

5.2 Onset and Collapse Detection

Two complementary envelopes are computed over the rectified signal $\mathrm{x}[n]$ using a half mains cycle sliding window maximum. This includes a forward looking (leading) envelope, used for collapse detection because it stays high right up until a real conducting burst has actually ended, and a backward looking (trailing) envelope, used for onset detection because it rises at the first true sample of a burst rather than lagging behind it.

Two calibration figures are computed once per waveform so every threshold below is self-calibrating rather than a hardcoded constant: A reference amplitude from the upper tail of the signal's own magnitude distribution, and a noise floor from the quietest blocks of the record. The derived thresholds are:

$$\text{off\_threshold} = \max(\text{noise\_floor\_v}, \min(0.5\text{ref\_amp}, \max(\text{off\_frac} \cdot \text{ref\_amp}, 6\sigma_{\text{noise}})))$$
$$\text{on\_threshold} = \max(\text{on\_frac} \cdot \text{ref\_amp}, 1.25 \cdot \text{off\_threshold})$$
$$\text{residual\_floor} = \min(\text{off\_threshold}, \max(\text{noise\_floor\_v}, 5\sigma_{\text{noise}}))$$

with `on_frac = 0.25`, `off_frac = 0.10`, and `noise_floor_v = 0.5` all locked in `config.yaml`. The `6σ_noise` term in `off_threshold` exists specifically so a noisy record still collapses cleanly instead of never quite dropping below a fixed fraction of amplitude.

Onset detection starts with a coarse crossing of `on_threshold` on the trailing envelope, guarded against a lone noise spike by also requiring a sustained forward looking rolling mean above `0.25 * on_threshold`, then a refinement stage that recovers sub-threshold conduction that may have begun slightly earlier, near a zero crossing, below the coarse detector's own confidence level.

Collapse detection searches forward from onset for the first point where the leading envelope stays below `off_threshold` for a full mains cycle. This persistence requirement is exactly what prevents an ordinary AC zero crossing mid burst from being mistaken for the actual end of conduction. It then walks the collapse point back to the last raw sample that actually crossed `off_threshold`. Note that the smoothed envelope lags the true crossing up to:

$$\frac{\sin^{-1}(\text{off\_frac})}{2\pi f_{\text{line}}}$$

This is a few hundred microseconds at typical noise levels. The detection method then walks forward again up to a quarter mains cycle to recover any sub-threshold residual tail down to `residual_floor`.

5.3 Verdict Logic

$$\mathrm{V}(t_{trip}) = \begin{cases} PASS, & t_{trip} \le 24.97\,ms + \Delta t_2 \\ NO\ TRIP, & t_{trip} > 100\,ms - 0.5\,ms - \Delta t_2 \\ FAIL, & otherwise \end{cases}$$

The verdict logic includes a dedicated NO_TRIP branch for cases where no envelope collapse is detected within the valid recording window, meaning the envelope never returns below `off_threshold`. The 0.5 ms `endpoint_uncertantity_s` margin applies only at the no trip boundary and only in the fail safe direction. In other words, a trip measured within that margin of 100 ms is treated as a NO_TRIP rather than risk underreporting and marking the device as slow. However, it is not applied at the pass limit, which stands at its literal configured value (MEAS-04). Sanity check results are never consulted inside this decision.

All but two of the six sanity checks are logged only; the two that gate a halt (K3 pretrigger current and record too short to trust a no trip conclusion) represent distinct safety conditions, not measurement uncertainty about the verdict itself.

5.4 Algorithm Implementation Details

5.4.1 The van Herk / Gil-Werman Sliding Window Maximum

Computing the maximum over every sliding window of width W across an N-sample array naively costs $\mathrm{O}(N \cdot W)$. At 1,000,000 samples per cycle, and 6,000 cycles in a campaign, with W on the order of a few thousand samples, that cost is not viable to run in a poll loop. The van Herk / Gil-Werman (vHGW) algorithm computes the same result in $\mathrm{O}(N)$ time total.

First, partition the array into contiguous, non-overlapping blocks of size W. Within each block, compute a prefix max (running max scanning left to right from the block's start). Then compute a suffix max (running max scanning left to right from the block's end). Each of these computations cost $\mathrm{O}(N)$ total across all blocks, since every sample is visited exactly twice. Then, for any window of width W starting at index i, the window's maximum is exactly:

$$\max_{j=i}^{i+W-1} x[j] = \max\left(\text{suffix}[i],\ \text{prefix}[i+W-1]\right)$$

This holds because $\text{suffix}[i]$ already covers everything from i to the end of i's own block, and $\text{prefix}[i+W-1]$ covers everything from the start of that same block through $i+W-1$. Together, they exactly span the window regardless of where the block boundary falls inside it. Every window query after the preprocessing pass is then $O(1)$.

The actual implementation (`ccid/analysis.py::sliding_max`) matches this algorithm directly. The array is zero padded to a multiple of `window`, reshaped into `(n_blocks, window)`, and `prefix`/`suffix` are computed with `numpy.maximum.accumulate` along each block row, resulting in a fully vectorized realization of the block decomposition above, not a Python-level loop over blocks. The per-window result is then `np.maximum(suffix[head], prefix[head + window - 1]` for every valid starting index.

Edge cases are handled explicitly. `window <= 1` returns the input unchanged, `window > array.size` is clamped to one block, and an empty array returns empty. The test suite checks `sliding_max` against a brute force $O(N \cdot W)$ reference implementation over many window sizes and array shapes.

Leading and trailing envelopes are not interchangeable. A forward looking (leading) envelope at index i can "see" a burst that starts after i, which is perfect for collapse detection, but wrong for onset detection, where it would let future energy contaminate the classification of an earlier silent sample.

5.4.2 The Cumulative Sum Finder

Both collapse persistence and onset confirmation reduce to the same underlying question: what is the first index where a Boolean condition holds for `persistence` consecutive samples, without a Python-level loop up to 1,000,000 samples?

The technique to answer this question is implemented by first inverting the Boolean array, taking its cumulative sum, and observing that for any width `persistence` starting at index i, the count of `True` values in the original `below` array over that window is zero exactly when:

$$\text{cumsum}[i + \text{persistence}] - \text{cumsum}[i] = 0$$

i.e. the zero occurrences of `high` in that span means every sample in the span was `below`. This turns "first sustained run" into one vectorized cumulative sum and compare pass, $O(N)$, with `np.flatnonzero` locating the first qualifying index.

5.4.3 Exact Arithmetic Onset Refinement

After the coarse onset detector finds a high confidence crossing at index `burst_index`, `refine_start_index` searches backward for genuine sub-threshold conduction that may have started earlier, using the leading envelope against `residual_floor`:

```
below = indices where envelope_lead[:burst_index+1] < residual_floor
candidate = min(burst_index, below[-1] + window)
```

`envelope_lead[i] < residual_floor` means, by construction of a leading sliding max, that every sample in the window `[i, i+window]` is quiet. The first index where that stops being true means exactly one new sample entered the window since the last confirmed quiet index, so that sample is guaranteed to be the first raw sample at or above `residual_floor` after the last confirmed quiet stretch.

The primary defect in V2 (version 2) of the algorithm was that this candidate was trusted directly, with nothing confirming it represented a real, sustained conduction rather than a single noisy sample. On real 8-bit BYTE encoded captures, an isolated sample crossing the low `residual_floor` by pure quantization noise was enough to drag the refined onset back to an unrelated point in the pretrigger buffer. And because the leading envelope's window can chain together multiple nearby noise samples' forward reaching "shadows," this was not a rare, isolated failure mode.

V3 (version 3) of the algorithm's fix added one further requirement before trusting a refined candidate. The raw samples at that point must actually stay above `residual_floor` for a short sustained run using the exact same `_first_sustained_low` primitive with a fixed confirmation length of 2 μs.

```
confirmed = _first_sustained_low(magnitude >= residual_floor,
confirmation, start=candidate, limit=burst_index
return burst_index if confirmed is None else confirmed
```

If nothing near the candidate satisfies that stricter confirmation, the algorithm falls back to the already trustworthy coarse `burst_index` rather than trusting an unconfirmed candidate. V1 and V2's exact original behavior is preserved unmodified for historical replay (MEAS-03). The fix is a new code path selected by `algorithm_version`, never an edit to the V1/V2 branches.

5.4.4 Reference Amplitude and Noise Floor Estimators

`reference_amplitude`: The median of the top ~5% of $|V|$ across the record, sized from the half cycle window. Not a plain maximum and not a percentile over the whole record (which collapses toward zero on a record that is mostly post trip silence).

`_noise_sigma`: The 10th percentile of a half mains cycle block RMS values. A silent block reads the noise directly. A conducting block reads roughly 0.7 times the burst amplitude. Taking a low percentile recovers the noise floor without first needing to know in advance which blocks are actually quiet.

Both feed the threshold formulas in Section 5.2 directly, so the same algorithm self-calibrates to whatever amplitude and noise level a given capture actually has, rather than relying on fixed constants tuned to one signal level.

5.4.5 Independent Cross Validation Method B: CUSUM Change Point Detection via a Closed Form Lindley Recursion

This method was used only in the offline cross check reported in Addendum I.

A CUSUM (cumulative sum) sequential change point detector accumulates a one-sided running statistic (the Lindley recursion):

$$W_i = \max(0, W_{i-1} + x_i)\,, \quad W_{-1} = 0$$

It also signals a change point where W_i first crosses a threshold. Because W_i depends on W_{i-1}, this looks inherently sequential. At 1,000,000 samples across 6,000 cycles, a per sample Python loop is roughly 6 billion sequential operations. This is naturally impractical to run.

The implementation (`analysis/deep/deep_analysis.py::_cumsum_recursion`) instead uses a closed form identity.

Let

$$C_i = \sum_{j \le i} x_j$$

Which is an ordinary cumulative sum.

Then let

$$m_i = \min(0, C_0, C_1, \ldots, C_i)$$

Which is the running minimum of C, including 0.

Then

$$W_i = C_i - m_i$$

This is provable by induction. W resets to 0 exactly where C dips to a new running minimum (since $C_i - m_i = 0$ there by definition of m_i). Between resets W grows by exactly the accumulated excess of C above its most recent minimum, which is precisely what the clipped recursion computes. The implementation is the two `numpy` cumulative operations (`np.cumsum`, `np.minimum.accumulate`), turning a sequential algorithm into two vectorized passes.

Once this detector is gated against the same physical amplitude floor Method A (the threshold crossing detector described above), it converges to Method A's exact answer on effectively every cycle in the campaign dataset. This makes it a weaker independent check than a fully separate method would be. Note that Addendum I reports it as such, rather than two methods independently agreeing.

5.4.6 Independent Cross Validation Method C: Sigmoid Curve Fitting with Covariance Derived Uncertainty

This method was also used only in the offline cross check reported in Addendum I.

Rather than detect a threshold crossing, this method fits a logistic/sigmoid function to the RMS envelope at each transition edge via nonlinear least squares (`scipy.optimize.curve_fit`):

$$f(t) = \text{floor} + \frac{\text{amp}}{1 + e^{-(t-\text{center})/\text{width}}}$$

This reports the fitted `center` as the transition instant. This is a fundamentally different endpoint definition than a threshold crossing. A sigmoid's center is the midpoint of a transition, while V3's own endpoint is a threshold crossing refined toward the true collapse. This difference is what produces Addendum I's reported systematic offset (Method C reads about 1.9 ms lower than V3 on average).

The reported uncertainty for each fit is the standard error of the fitted center, taken from the fit's covariance matrix:

$$\text{SE}(\text{center}) = \sqrt{\text{cov}_{22}}$$

Where cov_{22} is the diagonal element corresponding to the `center` parameter, as returned by the `curve_fit`'s `pcov`. This number describes the confidence in where the sigmoid's own center sits, given the data and the fitted model. Note that it should not be interpreted as an uncertainty on V3's `trip_time_s` under V3's own endpoint definition. Those are two different measurands sharing a unit, and Addendum I carries this distinction forward explicitly.

5.4.7 Circular Mean Hue Range Calibration Trick

This technique was used in `tools/calibrate_camera.py::propose_hue_range`, which proposes an HSV hue band for a given LED color from captured calibration footage. Hue is circular (0° and 360° are the same angle), so a plain percentile computed on raw hue degree values mismeasures any color band that straddles the wrap point. Red, in this system's default hue bands, sits at (345°, 15°).

The fix (`_circular_hue_range`) computes the circular mean of the sample angles. So not the arithmetic mean of the degree values, but the angle of the mean unit vector.

$$\bar{\theta} = atan2\left(\frac{1}{n}\sum_i \sin\theta_i\,,\ \frac{1}{n}\sum_i \cos\theta_i\right) \bmod 360°$$

It then rotates every sample so that $\bar{\theta}$ lands at exactly 180°, which is the point on the circle maximally distant from the 0°/360° discontinuity. This is so that a percentile window computed in this rotated space can never itself straddle the wrap point regardless of where the original cluster sat. The low/high percentile is taken in that rotated space, then rotated back by subtracting the same shift.

The implementation is as follows: `shift = (180 - mean_angle) % 360`, `shifted = (hues + shift) % 360`. The percentile is taken on `shifted`, then `(result - shift) % 360` to unrotate.

4.8 Other Implementation Notes

Atomic `runstate.json` Replacement: The new state is written to a `NamedTemporaryFile` created in the same directory as the target, fsynced, then moved into place with `os.replace(tmp, path)`. `os.replace` is atomic on POSIX only within a single filesystem. A temp file written into a different directory could span a filesystem boundary and lose that guarantee, which is why the temp file is deliberately created alongside the file it will replace rather than in a generic temp location. Any reader of `runstate.json` therefore always sees

either the fully old or new file and never a partially written one, even if the process is killed mid write.

Canonical Config Hash: The loaded configuration is serialized with `json.dumps(payload, sort_keys=True), separators=(“,”, “:”))` and hashed with SHA-256. `sort_keys=True` makes the hash independent of the key order in the source YAML file. The fixed compact `separators` remove incidental whitespace differences affecting the hash. The `monitoring` section contributes only the environment variable name used to resolve the Cronitor URL, never the URL or key itself, since no secret value is ever loaded into the config object in the first place.

5.5 Uncertainty

A first order uncertainty model for the reported trip time, in combined standard uncertainty form:

$$u(t_{\text{trip}}) = \sqrt{u^2(t_0) + u^2(t_{\text{end}}) + u^2(t_{\text{sample}}) + u^2(t_{\text{timebase}}) + u^2(t_{\text{algorithm}})}$$

Term	**Status**	**Value / Note**
$u(t_{\text{sample}})$	Known	$\approx dt/2$; $dt \approx 500$ ns
$u(t_0), u(t_{\text{end}})$	Known	Not a live uncertainty term for V3, ≈ 123 ms obvserved in V2
$u(t_{\text{algorithm}})$	Known	Method A/B: $\approx +2.4\ ms$ above V3 on average Method C: $\approx -1.9\ ms$ below V3 on average
$u(t_{\text{timebase}})$	Unsure	Oscilloscope calibrated 06/06/2026; Tech: JADAMS; Exact data unknown
ADC Quantization Step	Known	2.01005 V
`endpoint_uncertainty_s`	Known	0.5 ms, deliberate fail safe margin, not a general purpose uncertainty term applied throughout
Probe Accuracy	Unsure	10:1 Attenuation Ratio is a nominal spec, Oscilloscope calibrated 06/06/2026; Tech: JADAMS; Exact data unknown

The slowest recorded PASS and the fastest recorded FAIL in the 6,000 cycle campaign (See Addendum I) are separated by only 11.0 µs. This is smaller than several of the definitional offsets in the table above. This system's classification according to its own configured algorithm and thresholds is precise to that boundary. Furthermore, whether that classification also represents compliance with UL 2231-2 to the same precision is a distinct claim this report does not make, pending the timebase/probe calibration terms above.

6. Software and Data Architecture

Every cycle's data is committed to disk in a fixed order chosen specifically so a crash at any point leaves the run in a state that can always be cleanly recovered on the next start, without ever losing a completed cycle or double counting one that never finished (DAT-01)

The raw waveform, scope screenshot, camera gate frame, and per-cycle JSON sidecar are written and fsynced first. Only then does the CSV row get appended, `runstate.json` get atomically replaced, and an external liveness signal sent. A resume trusts nothing but `runstate.json`'s own record of the highest completed cycle number. Any artifact left behind by a cycle that never reached that point is an orphan and is discarded rather than trusted.

A halted run stays halted (OPS-02) and the run's own state file records why it stopped. A halt is sticky across restarts until explicitly overridden, so a device that has already failed a real test cannot be silently re-tested without a human being aware a halt occurred.

Additionally, every raw waveform is preserved in full (MEAS-02), not just the number the algorithm computed from it at the time, specifically so a later corrected algorithm version can be applied to historical data without repeating the physical test.

7. Failure Handling and Recovery

For most of this system's development, any halt at all whether it be a scope timeout, an unexpected exception, a disk-space fault, or a genuine device failure, ended the campaign outright and left it idle until a human noticed and manually resumed it. The resolution treats halt categories asymmetrically (OPS-01). Halts are sticky by design specifically so a device that has already failed a real test is never silently re-tested, and an indiscriminate auto retry everything policy would have defeated the purpose of that safeguard.

A NO_TRIP verdict gets a short leash of 3 consecutive occurrences before a human is required. Every other kind of halt not tied to a device verdict gets a longer leash of 5, on the reasoning that those are substantially more likely to be transient rig or software issues than evidence the device under test has actually failed. Either streak resets the moment a cycle completes without issue.

Camera and oscilloscope connections receive periodic and reactive maintenance (OPS-03). This includes a full stop/start on a fixed schedule every 50 cycles, as well as after 3 consecutive camera unavailable cycles. These refreshes are always strictly before mains close for the cycle they apply to, and while refresh failures are logged and swallowed, the underlying problem still surfaces through the cycle's own proper halt path. Neither auto retry nor equipment refresh can recover a scope connection already marked permanently unusable following an unrecoverable diagnostics failure.

8. Known Limitations

No auxiliary contact or voltage feedback: There is no electrical or software based way to confirm a contactor physically reached a commanded state. The software can only track what has been commanded.

Protective earth continuity is not independently and continuously monitored by any isolated sensing path: A fault would only be inferred indirectly, after the fact, through downstream consequences.

No confirmation from UL on their standard's published text: The measurement endpoint definition remains this project's own provisional reading and understanding of UL 2231-2.

No campaign level statistical acceptance criterion: What pass rate, across how many cycles, would constitute a satisfactory result is not defined anywhere. That judgement is left to my superiors at Legrand North and Central America – The Wiremold Company.

Auto retry and equipment refresh have been proven against a live repeat of the exact real hardware incident that motivated it: At the point this system entered its first large campaign, auto retry and refresh had been validated against realistic simulated failure conditions and passing regression tests, but the true incidents that prompted this feature have not been replicated as they are unknown (log files from earlier incidents lost due to power cycling).

9. Verification and Commissioning

9.1 Requirements and Verification Matrix

ID	Design Implementation	Verification Method	Result
SAF-01	GPIO active high, initialized inactive	`test_safety.py`	Pass
SAF-02	Single use `ChargingGateToken`, checked in `gpio_real.py`/`gpio_sim.py`	`test_safety.py`, fault matrix rows in `test_faultmatrix.py`	Pass

SAF-03	Interlock in both HAL implementations; blocks K1/K2 open while K3 closed	`test_safety.py`	Pass
SAF-04	`Sequencer._poll_acquisition_with_backstop`	`test_sequencer.py`	Pass
SAF-05	`ccid/safety.py::safe_off`, called from every halt/retry/completion path plus a redundant outer call in `_execute_campaign`	`test_safety.py`	Pass
MEAS-01	`_resolve_t0` precedence order; trigger instant never used as t_0	`test_analysis.py::TimeBaseTests`	Pass
MEAS-02	Raw .npz written before analysis; `tools/replay_waveform.py` reanalyzes without repeating the physical test	`test_tools_replay_waveform.py`	Pass
MEAS-03	`AnalysisVersion` enum; V1/V2 branches frozen and V3 added alongside	`test_analysis.py::VersioningTests`	Pass
MEAS-04	Sanity checks recorded but not consulted except the two gating checks	`test_analysis.py::SanityCheckTests`, `VerdictBoundaryTests`	Pass
MEAS-05	`analysis.endpoint_definition` frozen into the config hash	N/A	Open
VIS-01	Single use gate token gated on grand	`test_classify.py::ChargingGateTests`	Pass
VIS-02	`await_charging_gate` swallows every camera failure into a degrade and continue path	`test_classify.py`	Pass
OPS-01	`_run_campaign_with_auto_retry`	`test_main.py`	Pass (Simulated)
OPS-02	`RunRecorder.load_run_state`'s sticky halt check	`test_resume.py::test_resume_blocks_on_halt_without_override`	Pass
OPS-03	`Sequencer._maybe_refresh_equipment` and `_refresh_equipment`	`test_main.py`	Pass
DAT-01	Fixed commit order	`test_resume.py`	Pass
DAT-02	`AppConfig.canonical_hash()` checked on every resume	`test_config.py::test_hash_stable_across_key_order`; `test_resume.py::test_resume_blocks_on_config_hash_mismatch`	Pass
DAT-03	`config_hash` / `software_version` /	Inspection of any run's `cycles/<number>.json`	Pass

	`analysis_version` repeated into every per-cycle sidecar		

The software stack includes 364 automated tests, with 2 intentional skips, run in roughly 7 seconds. The two documented skips are properties that are either physically unverifiable from software alone (K1/K2 physically stuck closed), or owned by an external service this codebase does not reimplement (Cronitor missing heartbeat alerts). Please visit `docs/test-suite-guide.md` for more information on the test suite.

9.2 Pre Hardware Validation Ladder

The system was validated moving from pure software toward hardware one step at a time.

1. A full automated test suite on the Raspberry Pi itself
    a. This included a full simulated campaign through the software's own simulated HAL, including a deliberately induced crash partway through to confirm resume behavior recovers cleanly
2. GPIO check exercising the contactors without real fault current present
3. Oscilloscope check confirming connection/configuration/arming without requiring an actual trigger event.
4. Camera calibration passes against real footage of the EVSE's status LED.

The HAL split is what makes this staging possible. Each subsystem's mode flips from simulated to real independently, so one real subsystem can be validated in isolation with everything else still simulated around it. This staged approach caught two real problems well before any hardware was at risk, neither involving the electrical rig at all.

First, a required GPIO driver package installed system wide but invisible to the project's isolated Python virtual environment. The system would silently fall back to an experimental, less trustworthy GPIO backend. Interestingly, this failure mode would have looked like ordinary success on a quick manual check.

Additionally, the Raspberry Pi's own temporary file filesystem was found to be far smaller than the project's disk space safety guard required, causing every software only test to halt on a disk space fault despite ample real free space elsewhere on the same machine.

9.3 Notable Hardware Commissioning Issues and Investigations

Three investigations most shaped the current design. Each is summarized in this document. The full chronological record, including every dead end, wrong hypothesis, and intermediate tooling bug can be found in `docs/build-and-commissioning-issue-log.md`. For the scope

investigation specifically, the raw SCPI level log can be found in `docs/scope-trigger-debug-log.md`.

9.3.1 Oscilloscope No Trigger Investigation

Problem: For an extended period, every real energized cycle armed the oscilloscope, closed K3, and never triggered. The scope stayed armed, the trace stayed flat, and the cycle timed out and halted, on a rig whose contactors, camera, and analysis logic had all already been separately confirmed working.

Observed evidence:

- The operation condition register stayed at the "armed" value for the entire acquisition window
- No acquisition ever completed
- A diagnostic-only forced trigger eventually froze a large bipolar transient (~−167 V to +141 V) on the input, with the trigger event register confirming the comparator had never fired naturally.

Candidate causes investigated:

Candidate Cause	Finding	Effect on Final Design
`:TRIGger:MODE EDGE` never explicitly sent before edge parameters	Confirmed real defect	Fixed, but did not resolve the no trigger condition
`:CHANnel1:SCALe/OFFSet` sent before `:CHANnel`:PROBe`	Confirmed real defect	Fixed, but did not resolve the no trigger condition
`TRIGgerLNREJect` unsupported on this instrument	Confirmed unsupported	Command removed entirely
Background thread based diagnostics timeout	Confirmed unsafe	Replaced with PyVISA's native synchronous per-resource timeout. Any query failure permanently marks the connection unusable for the rest of the process.

Root Cause: A single status flag in the sequencer's forced diagnostic polling loop was used to mean two different things: "the diagnostic checkpoint ran" and "a trigger was actually forced." On exactly the branch where a real, natural trigger was correctly detected and forcing was

correctly skipped, the surrounding poll loop still permanently stopped checking for a successful acquisition. Some fraction of the cycles this entire investigation was built around had very likely triggered successfully on real hardware and been silently misreported as failures by the diagnostic tooling itself rather than any defect in the electrical measurement path.

Fix: Gate “stop polling for real completion” on whether forcing actually happened (`force_command_return_monotonic_s is not None`), not on whether the checkpoint merely ran.

Verification: Natural triggering became repeatable immediately after the fix. Confirmed with a 25-cycle campaign, followed by three further real campaigns (200-cycle, 483-cycle, and the final 5317-cycle campaigns) totaling 6,000 cycles with zero recurrence of the no trigger condition.

9.3.2 Vision Charging Gate Redesign

Problem: A real commissioning cycle halted on a stuck BOOTING vision gate timeout while an I was directly watching the EVSE's LED flash green.

Observed Evidence: A synthetic noisy frame reproduction confirmed the mechanism. The temporal window classifier could be tipped from a correct GREEN reading into a false BOOTING verdict by as few as 2 stray colored frames out of a ~45 frame observation window.

Root cause: The existing classifier's design requires the entire rolling window to agree before declaring a state is the the wrong tool for recognizing a flashing charging indicator, which necessarily produces dark and off-color frames between flashes.

Fix: A second independent policy (ChargingGatePolicy) authorizes the gate grant directly. It accumulates green frame timestamps (not window classifications), tolerating dark gaps, and grants once a minimum count (3) of green observations spans a minimum real elapsed time (3.5 s within a 6 s window). The original window classifier is unchanged and still used for diagnostic LED state reporting.

A second order defect in the fix itself was also found through testing. The first version of the new policy accepted 3 green frames within just 2 seconds, which the EVSE's own multi-color boot animation could satisfy on its own. This risked a fault injection against a unit that only appeared ready. This was fixed by requiring a 3.5 second span, plus clearing accumulated evidence on either a red or blue observation (not red alone), and treating dark/unreadable frames between flashes as neutral rather than as a reason to discard already accumulated evidence.

Verification: Validated by physically disabling K3's coil supply and running the real camera and real contactors against a simulated oscilloscope. The resulting gate typically takes 45–50 seconds to grant on a real device.

9.3.3 Auto Retry and Equipment Refresh

Problem: Two real on large unattended runs exposed the same gap from different directions: any halt at all ended the campaign outright, and the resulting idle until noticed period was the actual source of a multi day run appearing to "keep crashing." Separately, one real campaign halted at cycle 38 with controller:unexpected:ValueError. The Pi became unreachable shortly after, and nonpersistent journald lost the original traceback with it, leaving only a code review theory rather than a confirmed cause.

Evidence: A boundary condition timing defect was identified by a code review as the plausible cause of the cycle 38 halt, though the original incident could not be definitively confirmed against the lost traceback. Separately, cycles 482–484 of a different run all reported the camera unavailable immediately, traced to a USB re-enumeration event (/dev/video0 changed to /dev/video1).

Root cause: Due to the lost traceback, a root cause could not be established.

Fix: An indiscriminate "auto retry everything" policy would quietly defeat the sticky halt safety property (OPS-02). Asymmetric auto retry streak limits, plus periodic and reactive equipment refresh (OPS-03) for the camera-re-enumeration class of problem, and every unexpected exception now persists its own full diagnostic record (type, message, traceback, cycle state) directly to durable per-cycle storage, independent of whether the process's own log stream survives whatever happens next.

Two second order defects were caught by writing a regression test. An early version of the "did progress happen" check counted a committed NO_TRIP as progress, which would have silently reset the tighter NO_TRIP streak the instant one occurred. This was fixed to require more than one cycle within a single continuous `sequencer.run()` call, which only completion can produce.

Separately, retrying after certain halts risked reusing an already spent, single use per cycle charging gate token. This was fixed by always advancing past whichever cycle number was actually attempted, committed or not.

Verification: The auto retry mechanism engaged exactly once across the entire subsequent 6,000-cycle campaign (following the single NO_TRIP verdict) and resolved cleanly on its next

attempt. While this is evidence toward correctness under an actual repeat condition, it is not exhaustive proof. Equipment refresh's real world activation remains unconfirmed either way as no comparable camera incident recurred in the 5,317-cycle run that had refresh available to it, but the underlying event is rare enough in this dataset (one occurrence in 6,000 cycles) that non recurrence in a single subsequent run is not exactly strong evidence.

10. Conclusion

Project A.M.P.E.R.E. was developed to automate repetitive CCID trip time testing at a scale impractical for manual operation while preserving measurement integrity, operational safety, and full data traceability. The system described in this report integrates automated fault injection, oscilloscope based waveform acquisition, vision based charging state verification, and crash safe data persistence into a single platform intended for long duration endurance testing.

Its design requirements, architecture, and validation activities have been presented in Sections 2 through 4. The commissioning process identified several defects prior to campaign deployment, including issues affecting oscilloscope acquisition, vision based state detection, and automated recovery behavior. In each case, the resulting corrective action was incorporated into the system design and subsequently verified through testing. The final implementation therefore reflects not only the original design intent, but also the lessons learned during commissioning and validation.

The requirements matrix and validation evidence presented in this report establish that the system satisfies the majority of its stated objectives with documented, reproducible verification. A small number of items remain open, including confirmation of the measurement endpoint interpretation against the applicable standard and several deferred hardware monitoring enhancements. Those limitations have been explicitly identified and should be considered when interpreting results produced by the system.

The primary conclusion of this report is therefore limited in scope. The design, safety controls, measurement methodology, persistence architecture, and validation activities provide sufficient evidence that the system was suitable for large scale unattended endurance testing. The operational performance of the system during such testing, and the resulting CCID trip time data collected across thousands of cycles, are reported separately in Addendum I.

Appendix A: Configuration Reference

Section	Parameter	Value	Meaning
GPIO	k1/k2/k3	GPIO17/ GPIO27/ GPIO22	BCM pin driving each contactor
Vision	roi_x/roi_y/roi_width/roi_height	35/ 120/ 450/ 350	Calibrated pixel region containing the EVSE status LED
Vision	charging_green_window_s	6.0	Rolling window the charging gate policy considers
Vision	charging_green_required_frames	3	Minimum green frame count within that window before a grant is even considered
Vision	charging_green_min_span_s	3.5	Minimum real elapsed time those green frames must span
Camera	device_index	/dev/ccid_ camera	Stable, udev resolved camera path
Timing	cooldown_s	10	Rest period between an ordinary completed cycle and the next
Timing	cooldown_retry_s	60	Additional rest period before a retried cycle after a halt
Timing	boot_timeout_s	90	Maximum wait for the charging gate before timing out
Timing	k3_backstop_s	0.300	Hard software backstop on leakage injection contactor closure
Timing	pass_limit_s	0.02497	Trip times at or below this are a pass
Timing	no_trip_limit_s	0.100	Trip times at or above this are a no trip
Timing	equipment_refresh_interval_cycles	50	Scheduled scope/camera reconnect cadence
Timing	equipment_refresh_after_consecutive_camera _unavaliable	3	Reactive refresh trigger threshold
Modes	gpio_mode/scope_mode/camera_mode	real/ real/ real	Per subsystem HAL selection

Paths	`min_free_disk_gb`	2	Minimum free space before the persistence layer halts the run
Analysis	`algorithm_version`	V3	Which trip time algorithm version is active
Analysis	`line_frequency_hz`	60.0	Mains frequency assumed by the envelope algorithm
Analysis	`envelope_window_cycles`	0.5	Envelope smoothing window, in mains cycle
Analysis	`collapse_persistence_cycles`	1.0	How long the envelope must stay low before a collapse is accepted
Analysis	`endpoint_uncertainty_s`	0.0005	Fail safe margin applied only at the no trip boundary

Appendix B. Glossary

CCID: Charge Circuit Interrupting Device
EVSE: Electric Vehicle Supply Equipment; the charging unit the CCID protects and the rig energizes.
Control Pilot: the EVSE-to-vehicle communication signal that the spoof device emulates.
Spoof / EV Emulator: A static, always-on device presenting a fake vehicle presence to the EVSE, used so a full charge cycle can be exercised without a physical vehicle connected.
K1 / K2 / K3: The three contactors: K1 and K2 switch the L1 and L2 mains legs; K3 switches the leakage-injection path that produces the fault the CCID must clear.
DUT: Device Under Test; in this system, the CCID.
HAL: Hardware Abstraction Layer; the interchangeable real/simulated driver interface.
ROI: Region of Interest; the fixed pixel rectangle within a camera frame that the vision system actually classifies, rather than the full frame.
Trip: The CCID successfully clearing the injected fault within the allowed time window.
No Trip: The CCID failing to clear the fault within the allowed window.
Sticky Halt: The system's design property that a halted run remains halted across restarts until a human explicitly overrides it, rather than silently resuming.
t_0, t_{end}: The resolved onset and end instants of a fault event, whose difference is the reported trip time.

Appendix C. Hardware Reference

Contactors: Three TE Connectivity ECK100B Series independently driven single-pole contactors. 12 VDC nominal, approximately 0.462 A. Operate voltage at or below 9 VDC; maximum rated voltage 13.2 VDC; release voltage at or above 1.2 VDC.

- K1 (L1 mains, GPIO17 / physical pin 11)
- K2 (L2 mains, GPIO27 / physical pin 13)
- K3 (leakage injection, GPIO22 / physical pin 15).

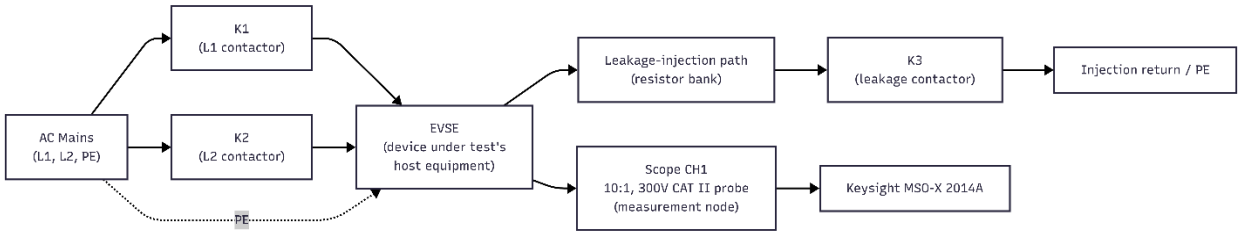
Driver: ZX-517 dual-MOSFET opto-driver board from its own isolated 12 VDC, 2 A supply. The ZX-517 boards are non-isolated and have no onboard flyback diode. All three driver boards share a single ground bond to the Raspberry Pi at one star point. GPIO outputs are active-high, initialized inactive.

Oscilloscope: Keysight MSO-X 2014A, firmware version 02.43.2018020635, connected over USB and driven via PyVISA with the PyVISA-py backend and PyUSB transport (no ARM-native VISA runtime exists for this platform). Positive edge trigger, channel 1 source, +20 V level, DC trigger coupling, noise-reject disabled (confirmed unsupported on this specific unit). RAW waveform acquisition mode in BYTE format with a one-million-point record, 50 ms/div centered timebase. Probe set to 10:1 passive attenuation, 300 V CAT II rating.

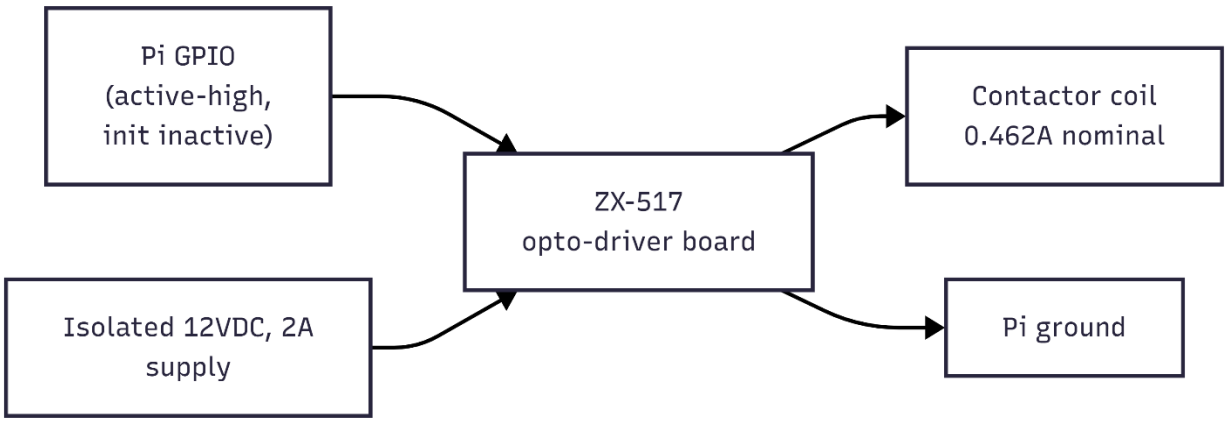
Camera: Logitech C270 USB webcam, YUYV 640×480 capture format, manual exposure locked (auto exposure and auto white balance both disabled/worked around), aimed at the EVSE's status LED. Resolved via a udev rule keyed to the camera's USB serial number to a stable `/dev/ccid_camera` symlink.

Controller: Raspberry Pi 4B, GPIO driven via `gpiozero` on the `lgpio` backend.

Single Line Electrical Diagram:



Contactor Driver Schematic:


Pi GPIO
(active-high,
init inactive)
Isolated 12VDC, 2A
supply
ZX-517
opto-driver board
Contactor coil
0.462A nominal
Pi ground

Appendix D. Detailed Technical Reference

The full commissioning history and detailed technical reference is preserved in multiple documents in the codebase repository's documents folder.

1. `docs/system-overview.md`
    a. The big picture and explanations on how subsystems fit together.
2. `docs/sequencer-and-state-machine.md`
    a. Covers `ccid/sequencer.py`, `states.py`, `safety.py`
3. `docs/trip-time-analysis-algorithm.md`
    a. Covers `ccid/analysis.py`, `forced_diagnostic_analysis.py`
4. `docs/hardware-abstraction-layer.md`
    a. Covers `ccid/hal/*`
5. `docs/vision-and-charging-gate-classification.md`
    a. Covers `ccid/classify.py`
6. `docs/persistence-and-recovery.md`
    a. Covers `ccid/recorder.py`, `config.py`
7. `docs/cli-lifecycle-and-monitoring.md`
    a. Covers `tools/*.py`
8. `docs/test-suite-guide.md`
    a. Covers the entire 364 item test suite
9. `docs/campaign-results-index.md`
    a. The bridge between the technical reference and the actual 6,000 cycle campaign data.
10. `docs/offline-campaign-analysis.md`
    a. The `analysis/` pipeline that was used to turn raw cycle data into distributions, verdict breakdowns, and plots from the committed V3 numbers. Includes descriptions of the exploratory algorithms (Method's B and C)
11. `docs/deployment-and-operator-runbook.md`
    a. An instruction set so that an operator knows what steps to take for booting up a fresh Pi, to perform the software validation ladder, and the full per-campaign preflight/run/troubleshooting checklist.
12. `docs/build-and-commissioning-issue-log.md`
    a. The narrative account of every struggle across the project, consolidated from daily logs that I took as I progressed through the project.
13. `docs/scope-trigger-debug-log.md`
    a. The raw SCPI level record of the oscilloscope no trigger investigation specifically, as this was a major roadblock within the project. It includes 15 detailed entries.

ADDENDUM I: 6,000 CYCLE ENDURANCE TEST

Executive Summary

Project A.M.P.E.R.E., described in full in the accompanying System Design and Validation Report, was built to exercise a CCID's trip time behavior across a far larger number of cycles than a manual test bench can practically produce. This addendum reports what happened when a combined 6,000 cycle endurance campaign ran. It produced 5,883 passing verdicts, 116 fails, and a single no trip out of 6,000 cycles.

This document covers the campaign's operational structure and preflight procedure, the incidents that actually occurred during the run itself, the resulting trip time distribution and its behavior over the course of the campaign, a reconstruction of when the system's auto-retry logic actually engaged, several observations that were not specifically anticipated going in, and an independent, non-authoritative cross check of the recorded verdicts using a second analysis method. Consistent with a deliberate design decision described in the System Design report, no campaign level pass/fail acceptance criterion is proposed or implied anywhere in this document.

1. Test Objective and Scope

Where the System Design and Validation report specifies what Project A.M.P.E.R.E. is and how it was built and validated to be trusted with a large unattended campaign, this addendum reports what actually happened the one time it was pointed at the scale it was built for. It describes how the 6,000-cycle campaign was structured and run, what incidents occurred during execution, what the resulting tri -time data looks like, and what offline analysis finds when pointed at the same raw waveforms.

This document assumes the System Design report as prerequisite reading and does not re-derive anything specified there. This includes the physical rig, safety interlocks, trip time measurement methodology (including every algorithm's exact implementation), the vision based charging gate logic, and the commissioning process that validated all of it are covered there and referenced here by section. This addendum exclusively covers the campaign's execution and the incidents that arose during that execution.

2. Test Article and Configuration

The system under test was JoinTech’s Gen II EV Charger. Its part number is JNT-EVM003/48AC/01C/SR/RF/WF/4G. Its unit sample number is U1W126800002, referred to as Unit ‘A’. Every configured threshold relevant to interpreting this campaign’s results, such as the pass/no trip limits, the analysis algorithm version, GPIO pin assignments, and the full timing configuration is listed in the System Design and Validation Report’s Appendix A. Any differences in configuration will be highlighted specifically in this report.

3. Test Runs and Data Provenance

3.1 Reasoning Behind Three Separate Runs

The 6,000 cycles reported in this addendum were not produced by one unbroken run. They come from three separate campaigns, run on different days, whose target cycle counts happen to sum to exactly 6,000: a 200 cycle run, a 483 cycle run, and a 5,317 cycle run. This is stated plainly because it materially changes how several results in Section 5 should be read. Most importantly the trip time over time analysis, which cannot treat the combined dataset as one continuous timeline without acknowledging the seams between attempts. The intended meaning of "6,000 cycles complete" for this project was always that the CCID be exercised across 6,000 cycles in total, not that it survive 6,000 uninterrupted repetitions in a single sitting, and the data reflects that intention rather than falling short of a stricter one.

Of the three, the 5,317 cycle and 200 cycle runs ran to their target with no halt and no evidence of a process interruption anywhere in the middle. The 483 cycle run was initially targeting 5,800 cycles, and was manually stopped well short of that target once camera unavailable behavior was noticed on its 482nd and 483rd cycles, and was itself not a single continuous sitting. This is not to claim that the 483 cycle campaign is any less usable, it just means that it should not be read as 483 consecutive cycles any more than the full campaign should be read as 6,000 consecutive ones.

3.2 Campaign Identifiers and data Provenance

Run ID	Target	Completed	Pass	Fail / No Trip
`200_v3_real_20260813T131932Z`	200	200	197	3
`5800_v3_try2_20260813T195018Z`	5800	483	471	12
`5317_v3_real_20260817T143315Z`	5317	5317	5317	102

The chronological order in which these three campaigns actually ran, by their own start timestamps: `200_v3_real` (August 13, 13:19 UTC), then `5800_v3_try2` (August 13, 19:51 UTC), then `5317_v3_real` (August 17, 14:33 UTC). This is the order used everywhere a single combined timeline is presented, since it reflects the sequence in which the underlying cycles were actually produced.

Every cycle used in this addendum's analysis carries its original source run ID and cycle index alongside a newly assigned position in the combined timeline (`global_cycle_index`, 16000),

so any statistic or plot here can be traced back to the exact source file it came from. All three campaigns ran under a configuration whose measurement relevant values were constant. This means the pass and no trip time limits, the analysis algorithm version, and every threshold the analysis algorithm itself uses were identical. The only configuration differences between them are operational rather than measurement relevant (the mechanism used to resolve the camera's device path, and the periodic/reactive equipment refresh behavior was only added after the 200 and 483 cycle campaigns had already run). Combining the three into one dataset for analysis purposes does not mix data collected under different measurement definitions.

The underlying data directory this addendum draws from also contains a large number of additional run directories from earlier commissioning and debugging work. None of those are campaign data, and none are included in any figure or statistic reported here.

4. Preflight and Execution Procedure

The preflight sequence executed before each of the three real campaigns follows `docs/deployment-and-operator-runbook.md` directly.

4.1 Physical and Repository Preflight

With EVSE mains and all three 12 V contactor coil supplies off, visual inspection of the setup includes:

- Ensuring protective earth connections are intact
- Ensuring K1/K2/K3 wiring and driver channel connections are verified (K1 = L1/GPIO17, K2 = L2/GPIO27, K3 = leakage/GPIO22)
- Ensuring probe attenuation switch confirmed at 10x on the approved measurement node
- Ensuring that the camera is mechanically secure and aimed at the EVSE LED
- Ensuring that there are no loose conductors or visible damage
- Ensuring that emergency disconnect is accessible
- Ensuring that the test area is restricted during energized operation

On the Pi itself:

```
cd ~/ccid_automation && source venv/bin/activate
git status --short
git log -3 --oneline --decorate
pgrep -af "python.*ccid.main" || echo "No active CCID process"
df -h "$HOME"
```

4.2 Software Test Gate

```
mkdir -p ~/ccid_test_tmp
TMPDIR="$HOME/ccid_test_tmp" env -u CCID_SCOPE_RESOURCE python -m
unittest discover -s tests -p 'test_*.py'
grep -n "algorithm_version" config.yaml
```

A failing test suite or a algorithm version other than v3 is disqualifying for a real hardware campaign.

4.3 Camera, Contactor, and Oscilloscope Preflight

Camera:

```
v4l2-ctl --list-devices
v4l2-ctl --device=/dev/video0 --set-
ctrl=auto_exposure=1,exposure_time_absolute=60
v4l2-ctl --device=/dev/video0 --get-
ctrl=auto_exposure,exposure_time_absolute
v4l2-ctl --device=/dev/video0 --get-fmt-video
```

A warmed frame (≥ 3 s of active reads) is captured and inspected. Ensure that the full LED is visible, it is centered inside the configured ROI, and the exposure is not washed out on green.

Contactors:

```
python -m tools.gpio_selftest exercise --contactor K1 --real --i-
understand-this-energizes-hardware --pulses 1 --hold-s 0.5 --cooldown-
s 1
python -m tools.gpio_selftest exercise --contactor K2 --real --i-
understand-this-energizes-hardware --pulses 1 --hold-s 0.5 --cooldown-
s 1
python -m tools.gpio_selftest exercise --contactor K3 --real --i-
understand-this-energizes-hardware --pulses 1 --hold-s 0.2 --cooldown-
s 1
```

Each contactor is confirmed to pull in once and release cleanly. Listen carefully for any buzzing or sticking.

Oscilloscope:

```
lsusb | grep -i "0957:1798"
export CCID_SCOPE_RESOURCE="USB0::2391::6040::MY58100795::0::INSTR"
python -m tools.scope_bench identify --real
python -m tools.scope_bench configure –real
```

The SCPI error queue is polled directly and confirmed clean. `:SYSTem:ERRor?` should return `+0, “No error”` before proceeding.

4.4 Network and Monitoring Preflight (For Autonomous Runs)

If an unattended campaign is being conducted at the West Hartford location for the Wiremold Company, plug in the Verizon USB tethered hotspot rather than connecting to the secure WiFi channel. To confirm the dedicated Verizon USB-tethered hotspot is the active route:

```
ip -brief address show usb0
ip route get 8.8.8.8
```

Ensure `CCID_CRONITOR_URL` is confirmed present in `/etc/default/ccid-automation`, and a normal heartbeat, a failure heartbeat, and recovery are each verified before the run starts, since the Cronitor monitor is known to auto recreate itself after deletion and its lifecycle is not assumed correct from a prior campaign. Additional, more robust, heartbeat monitoring platforms are available, I simply used Cronitor because it was free and quick to use.

4.5 Execution

Two invocation patterns exist, both calling the same underlying command:

```
python -m ccid.main --config config.yaml start --target-cycles <N> --
run-id <RUN_ID>
```

A supervised run executes this directly over an SSH session that must remain open for the run's duration.

An autonomous run wraps the identical command in a transient systemd-run unit tied to a freshly generated, never-reused run ID, so it continues if the SSH session disconnects.

```
sudo systemd-run --unit=<unit-name> --uid=ccid --gid=ccid \
  --working-directory=/home/ccid/ccid_automation \
  --property=EnvironmentFile=/etc/default/ccid-automation \
  --setenv=TMPDIR=/home/ccid/ccid_test_tmp --collect \
```

```
/home/ccid/ccid_automation/venv/bin/python -m ccid.main \
--config config.yaml start --target-cycles <N> --run-id <RUN_ID>
```

Neither pattern differs in what runs, only in what happens if the operator disconnects. A third always enabled systemd service also exists in this project's deployment configuration, but has never been used to produce any of the three real campaigns in this addendum, or any other real campaign to date. Every real run has used one of the two patterns above.

Once started, nothing about scope or camera controls is touched manually during the run, and a halt is never resumed automatically without a human first reading why it happened.

4.6 Post Run Verification

The following was applied to each of the three runs once fully de-energized, before treating its results as trustworthy:

```
python -m ccid.main --config config.yaml status --run-id <RUN_ID>
```

This command was to ensure that last_completed_cycle matches the target or the expected halt point, halt_reason is null for a normal completion, and PASS/FAIL counts are plausible.

```
cat runs/<RUN_ID>/cycles.csv
```

This command was to ensure that trip time strictly positive, that t_end >= t0, analysis_version = v3, every waveform sanity check is true, the camera state displays CHARGING at gate grant, and that there are no unexplained degraded_flags.

```
find runs/<RUN_ID> -maxdepth 3 -type f -printf '%p %s bytes\n' | sort’
```

This command was to ensure that the expected artifact was set present for every completed cycle (`cycles/<n>.json, waveforms/<n>.npz, images/<n>_scope.png, images/<n>_green.jpg`). No cycle's data, including a failed or otherwise invalid one, was discarded during this process for any of the three runs.

5. Results

All figures in this section are read directly from the already committed verdicts and trip time values the analysis algorithm produced. Nothing here recomputes, corrects, or reinterprets a verdict. No campaign level pass/fail acceptance judgment is made anywhere in this section and the numbers are reported only as they are.

5.1 Verdict Summary with Confidence Intervals

Across the combined 6,000 cycles:

Verdict	Count	Proportion	95% Wilson CI
PASS	5,883	98.05%	(97.668%, 98.370%)
FAIL	116	1.933%	(1.614%, 2.314%)
NO TRIP	1	0.017%	(0.003%, 0.094%)

Wilson score interval $z = 1.95996$ for a two sided 95% interval chosen over the simpler normal approximation because it stays well behaved at the extreme proportion the single NO TRIP represents, where a normal approximation interval can extend below zero.

These three categories are kept separate throughout this addendum rather than folded together, since the run state record only ever tracks a single combined "not pass" count internally, which would otherwise obscure that all but one of the non-passing cycles in this entire campaign were ordinary FAILs rather than the more serious NO_TRIP condition.

5.2 Trip Time Distribution

Across the 5,999 cycles with a recorded trip time (the single NO_TRIP cycle has none, by definition):

- Mean: 16.77 ms
- Standard deviation: 4.07 ms

Distribution shape:

- Minimum 7.69 ms
- 5th percentile: 11.55 ms
- Median: 16.53 ms
- 95th percentile: 24.25 ms
- 99th percentile: 25.15 ms
- Maximum: 25.56 ms

The distribution is not unimodal. A smoothed density estimate shows three distinct peaks, at roughly 12.0 ms, 17.7 ms, and 23.0 ms, with consistent spacing of approximately 5.3 to 5.6 ms between them. That spacing does not cleanly correspond to either a half (8.33 ms) or a quarter (4.17 ms) mains cycle at 60 Hz, so it is not readily explained by a simple zero crossing locked tripping mechanism. Reported here as an open observation (See Section 10).

Because the combined 6,000 cycle timeline is three separate attempts laid end to end rather than one continuous run, a linear drift model,

$$t_i = \beta_0 + \beta_1 i + \epsilon_i,$$

was fit two ways. Once across the full combined timeline and once restricted to the 5,317 cycle run alone, since that is the only one of the three attempts both long enough and internally continuous enough for a same sitting drift trend to mean anything on its own.

Scope	n	$\widehat{\beta_1}$	r	p
5,317 cycle run only, PASS	5,215	−0.0225 µs/cycle	-0.0088	0.527
5,317 cycle run only, PASS + FAIL	5,316	−0.0303 µs/cycle	-0.0114	0.406
Full combined timeline, PASS	5,883	−0.0118 µs/cycle	-0.0052	0.690
Full combined timeline, PASS + FAIL	5,999	−0.0201 µs/cycle	-0.0086	0.508

Neither analysis found a statistically meaningful trend. Every r is under 0.012 and every p is well above 0.4. No slow upward creep in trip time, which would indicate a device or measurement chain degrading over the course of the campaign, is present in this data at a level distinguishable from ordinary cycle to cycle noise.

5.3 Boundary Results

All 116 FAIL verdicts fall inside the 24.97 to 100 ms band by definition, and every one sits close to the pass limit end of that band. The closest FAIL to the pass limit recorded 24.9715 ms (0.0015 ms above the line), and even the FAIL furthest from the pass limit recorded only 25.5580 ms, which is 74.44 ms clear of the no trip limit on the other side. There is no cycle in this dataset describable as a near miss no trip that happened to clear just in time. In fact every FAIL observed was a clearing that took just slightly longer than the pass threshold allows.

Cycle	Trip Time	Margin
Slowest PASS	24.9605 ms	9.5 µs below the 24.97 ms pass limit
Fastest FAIL	24.9715 ms	1.5 µs above the pass limit
Slowest FAIL	25.5580 ms	74.44 ms below the 100 ms no-trip limit

The slowest-PASS/fastest-FAIL gap (11.0 µs) is smaller than several of the definitional offsets discussed in Section 8 and in the System Design report's uncertainty budget.

5.4 Near Boundary Endpoint Sensitivity

The 11.0 µs gap in the previous section motivated a direct inspection of both waveforms' raw samples around their reported `t_end`.

By the time each waveform reaches its reported `t_end`, the signal has already decayed to the scope's own ADC quantization floor. The samples bounce between 0 V, 2.01 V, and 4.02 V, exactly 0, 1, and 2 counts of this scope's 2.01005 V per count resolution. There is no visibly decaying current left at that point.

The production algorithm's final refinement step walks the collapse point forward, past the initial envelope threshold crossing, looking for sub-threshold residual conduction down to `residual_floor`. This was done since real current sometimes tapers off gradually rather than stopping instantly. `residual_floor` is derived from each waveform's own estimated noise level (`5 × noise_sigma`) and is not guaranteed to sit meaningfully above the ADC's own quantization step. For both of these two specific cycles, `residual_floor` computes to approximately 4.0 V, which is almost exactly 2 ADC counts. At that level, an ordinary noise sample landing on 4.02 V purely by chance is indistinguishable from genuine residual current. This means that the exact instant chosen as `t_end` for two of the most boundary defining cycles in this entire campaign is plausibly landing on noise, not signal.

To bound how many cycles are structurally exposed to the same mechanism, cycles were counted by distance from the 24.97 ms pass limit:

Distance from Pass Limit	Cycle Count
Within 50 µs	23
Within 100 µs	48
Within 500 µs	239
Within 1 ms	361

Even in an implausible worst case where every one of the 239 within 500 µs cycles being mislabeled by this mechanism, the campaign's headline pass rate would only move by a few tenths of a percent. The defined confidence intervals already comfortably absorb a shift of that size. The distribution shape, the drift analysis, and the single unambiguous NO_TRIP are unaffected, since none of those conclusions rest on margins anywhere near this scale. What is not fully established however is the individual verdict on any specific cycle within roughly half a millisecond of the 24.97 ms line. For such a cycle, the honest statement is that it is probably

correctly classified, but a real mechanism exists that could be nudging a `t_end` on either side of the line, and it has not been fixed or revalidated against this campaign's data.

The recommended follow up is to require `residual_floor` to sit a firmer, quantified margin above the ADC's own quantization step rather than allowing it to land almost on top of it, and/or require the forward residual tail search to confirm more than one isolated sample before extending `t_end`.

5.5 Per Run Breakdown

5.6 NO TRIP Instance

The single NO_TRIP verdict occurred at cycle 3,115 of the 5,317-cycle run (global cycle index 3,798), recorded 2026-08-19T19:32:45 UTC. Its analysis record shows no envelope collapse anywhere within the full captured window. The fault current was still conducting at the end of the record and nothing about that waveform's own signal quality figures (reference amplitude 166.83 V, noise sigma 0.977 V) stands out as anomalous relative to the rest of the campaign. There is no indication this was a capture or equipment problem either. The raw waveform itself (Appendix B) shows sustained AC conduction continuing across multiple full mains cycles past t_0 with no sign of an approaching collapse.

6. Measurement Chain Configuration and Verification

Two configuration questions about the instrumentation chain (the oscilloscope's timebase setting and the channel's coupling mode) were raised by Dr. Islam and investigated during the preparation of this addendum.

6.1 Timebase (20 ms/div vs. 50 ms/div)

The oscilloscope's timebase (`timebase_scale_s_per_div`, 50 ms/div per the locked `ScopeSettings`) sets the total time window a single acquisition spans. For this scope's 10 division display, 50 ms/div gives a 500 ms captured window. Separately, `waveform_points` is configured to `MAXimum`, meaning the scope always transfers as many points as its memory depth allows for whatever window is showing. The actual per-sample time spacing is therefore:

$$dt = \frac{\text{captured window}}{\text{points captured}}$$

At 50 ms/div, the preamble reports exactly 1,000,000 points and `x_increment` = 5×10^{-7} s (0.5 µs, a 2 MHz effective sample rate), matching $500\ \text{ms}/1{,}000{,}000 = 0.5\ \mu\text{s}$ exactly. This confirms the sample interval this campaign's data was actually captured at.

This oscilloscope is memory depth limited, not acquisition speed limited, at both 50 ms/div and 20 ms/div. Switching to 20 ms/div (a ~200 ms window) would therefore shrink dt to roughly 0.2 µs rather than widen it. That sounds like an improvement, but the verdict decision only uses $dt/2$ as a quantization margin at the pass/no trip boundaries. This equates to 0.25 µs at the current 0.5 µs sample interval, against a 24.97 ms threshold. Shrinking that to 0.1 µs is a difference of a tenth of a microsecond, three orders of magnitude smaller than the algorithm's own known endpoint refinement bias. Sample clock resolution is not the limiting factor on this system's measurement precision.

Halving the captured window from ~500 ms to ~200 ms also comes with a cost. With the centered timebase this system uses, that shrinks the post trigger portion from roughly 250 ms to roughly 100 ms, and the no-trip limit itself is 100 ms, measured from t_0. A ~100 ms post trigger record risks falling under the window the `SANITY_RECORD_SPANS_NO_TRIP_LIMIT` check exists specifically to guarantee. Shrinking the pretrigger portion the same way would also weaken the pre trigger leakage (K3-stuck-closed) check's own baseline.

Switching to 20 ms/div would provide a resolution gain, but three to four orders of magnitude smaller than this system's actual sources of endpoint uncertainty, while the window reduction directly erodes the safety margin the current setting was deliberately chosen to provide. ms/div` is also a horizontal/time setting exclusively and has no bearing on voltage/amplitude precision, which is governed by V/div and the scope's 8-bit ADC.

6.2 Channel Coupling (AC vs. DC)

Channel 1 is configured with AC coupling, which inserts a high pass filter between the probe and the digitizer. This matters most for the campaign's single NO_TRIP cycle, whose conduction persists for nearly 250 ms.

That cycle's raw waveform was checked directly for amplitude droop. RMS amplitude was measured in successive 10 ms windows across the full conducting period, from onset to the end of the record, and no droop was found. Amplitude cycles between roughly 108 V and 125 V repeatedly throughout. The pattern at the start of the window ($t \approx 0$ to 50 ms) and at the end ($t \approx$ 200 to 240 ms) is essentially identical. A real AC coupling droop over nearly 15 mains cycles of sustained conduction would show a clear downward trend, but none is present.

Note that this is evidence from one cycle, not a full characterization of this scope's AC-coupling corner frequency. Within that scope, AC coupling does not appear to have measurably distorted this campaign's results. DC coupling remains the more textbook correct choice for a one-shot transient capture in general and is flagged as a future configuration change to consider.

7. Retry and Streak Analysis

The asymmetric auto retry behavior engaged once across the entire 6,000 cycle campaign, following the single NO_TRIP above. The gap before the next cycle ran roughly 50.0 seconds longer than that run's typical cadence, closely matching the campaign's configured 60 second retry cooldown, immediately after which a normal cycle resumed. No cycle index numbering gap appears anywhere in any of the three campaigns' records, which ensures that nothing that halted before a cycle could even be committed ever occurred.

Neither streak limit came close to being exhausted. The NO_TRIP streak reached a maximum of one, since only a single NO_TRIP occurred and it was not adjacent to another one. The other streak, covering rig faults, unexpected controller exceptions, and persistence failures, never advanced past zero. This is consistent with Section 9.4’s finding that all six waveform sanity checks passed on every one of the 6,000 cycles.

8. Independent Reanalysis

8.1 Motivation and Methodology

This section reports what happened when two structurally distinct offline detection methods were pointed at the same 6,000 raw waveforms and asked if a differently designed approach ever disagreed with the production algorithm's committed verdict.

Three detectors were run, kept structurally separate from the production algorithm. Method A is an independent envelope threshold detector with its own noise floor and reference amplitude estimators. Method B is a CUSUM sequential change point detector on windowed signal power. Method C fits a sigmoid curve to each transition edge and reports the fitted center. The exact mathematics and implementation of Methods B and C are specified in the System Design report Sections 5.4.5 to 5.4.6 and are not repeated here. This section reports only the results of applying them to the campaign data. All three ran successfully across all 6,000 waveforms with no failures.

One methodological result is worth restating. Method B, once modified during development to require confirmation against the same physical amplitude floor Method A uses ended up landing

on effectively the same answer as Method A on nearly every cycle in this dataset. Method C remained the truly distinct comparison point throughout.

8.2 Findings

Both alternative methods disagree with the production algorithm's trip time numbers by a small amount. The mechanism for each offset is a definitional difference in endpoint convention rather than a disagreement about the underlying physical event.

Method	**Mean Offset from Production**	**Median Offset from Production**	**Standard Deviation**	**r vs. Production**
A/B (Threshold Based)	+2.407 ms	+2.393 ms	1.457 ms	0.9363
C (Curve-fit Midpoint)	-1.897 ms	-1.988 ms	2.025 ms	0.9955

The threshold based methods read higher because they lack the production algorithm's raw sample endpoint refinement step and report the coarser envelope threshold crossing directly. The curve fit method reads lower because a fitted transition's center point is a different definition of "when did it end" than a threshold crossing is.

The offset correction subtracts each method's own median offset from the production algorithm's value computed once across all 5,999 waveforms with a recorded trip time and applied uniformly. No boundary cycles were excluded when computing the offset, and the same offset is applied regardless of a cycle's distance from either configured limit.

After that correction, the primary question was: does either method ever land on the opposite side of the 24.97 ms boundary from the production algorithm? A small number of cycles do flip (27 of 5,999 under the threshold based methods, 62 of 5,999 under the curve fit method) but every one of them was already a production algorithm trip time within roughly 1.45 ms of the boundary to begin with, and no cycle far from the boundary flips under either method. This is consistent with what any method carrying a few milliseconds of its own spread be expected to do when compared against a boundary this tight. It is simply expected boundary band jitter, not a set of mistimed cycles.

8.3 Limitations Found

The threshold based methods' fixed ~8.33 ms smoothing window produces a substantially larger bias, roughly +5.5 ms rather than the usual +2.4 ms, for the very fastest trips in this campaign's data since an event that short is comparable in duration to the window used to smooth it. This is a

structural weakness of a fixed window approach at the fast end of the distribution, and stands as an evidenced point in favor of the production algorithm's raw sample refinement design, which is not limited in the same way.

Separately, a scatter comparison of the alternative methods' trip times against the production algorithm's own numbers shows a banded, stepped structure rather than smooth scatter around the offset line. Both observations are recorded here as open questions in Section 10.

9. Anomalies and Interruptions

9.1 Equipment Refresh Activations

The periodic and reactive equipment refresh behavior was only present in the configuration used for the 5,317 cycle run. It had not yet been introduced when the 200 and 483 cycle runs took place. Within the 5,317 cycle run itself, neither the scheduled or reactive refresh ever visibly affected a cycle, as no degraded mode flag of any kind appears anywhere in that run's cycles. This is consistent with the refresh mechanism doing its intended job silently in the background, but since a silent absence of problems does not by itself prove a mechanism was exercised, one cannot be certain as direct confirmation that it fired and worked.

The one real camera unavailable incident anywhere in this campaign's data happened in the run that preceded equipment refresh's introduction (`5800_v3_try2`), using the older fixed wait degraded handling behavior. No comparable incident occurred in the later 5,317 cycle run, which did have equipment refresh available to it. That absence is suggestive of the fix having done its job, but the underlying event is rare enough in this dataset (one occurrence in 6,000 cycles) that its non-recurrence in a single subsequent run is not strong evidence of anything.

9.2 Correlations and Secondary Observations

A moderate correlation was found between a cycle's trip time and two of the diagnostic figures the analysis algorithm computes from each waveform.

Pair	**r**	**p**	**R^2 (variance explained)**
Trip time vs. reference amplitude	0.4447	$2.27/\times 10^{-289}$	19.8%
Trip time vs. noise level	-0.5873	Underflows to 0 in double precision ($p < 10^{-300}$)	34.5%
Reference amplitude vs. cycle index	0.134	≈ 0	1.8%

Both trip-time correlations are statistically overwhelming at $n = 5{,}999$ as expected, since p values shrink with sample size regardless of effect size. But the more informative figures are the R^2 values. Reference amplitude explains about a fifth of trip time variance, and noise level about a third, leaving the majority of variance unexplained by either. Neither should be read as necessarily physical without qualification. The analysis algorithm's own onset and collapse thresholds are themselves derived from each waveform's reference amplitude, so part of this relationship may reflect the detection algorithm's own sensitivity to signal amplitude rather than a change in how the device under test behaves at different fault current levels. Separately, reference amplitude itself varies across a narrow, quantized range consistent with ordinary mains voltage fluctuation rather than equipment drift, and its own correlation with cycle index is weak (r = 0.134, $R^2 = 1.8$).

No meaningful pattern was found in trip time or pass rate by hour of day.

9.3 Sanity Checks Summary

Every one of the analysis algorithm's six waveform sanity checks:

1. Signal presence
2. Absence of pre-fault leakage current
3. Sufficient record length to rule out a no trip
4. Onset timing consistency
5. A clean final collapse
6. No trip persistence where relevant

passed on all 6,000 cycles in this campaign. Every cycle's onset time was resolved from the waveform's own detected onset (`t0_source = "detected_onset"` on all 6,000 rows). This is a clean result for the capture and analysis pipeline's own internal consistency across the full campaign, independent of and separate from the trip time results themselves.

10. Discussion

10.1 System Reliability Assessment

The rig completed 6,000 fault injection cycles across three attempts with no safety incidents. The measurement and analysis pipeline behaved with complete internal consistency across the entire campaign, and the one auto retry event and the process pauses within the 483 cycle run were each recovered from cleanly, without losing, duplicating, or misattributing a single cycle's data.

The independent cross-check in Section 8 found no evidence that the production algorithm's real time safe design is missing something a heavier offline method would catch, and the inspection in Section 5.4 found one mechanism that could affect verdicts for cycles very close to the pass/fail boundary.

What this campaign does not support saying is whether a 98.05% pass rate, or the specific pattern of FAILs observed, constitutes an acceptable result for the device under test. That judgement is left to qualified engineers and compliance decision makers within Legrand North and Central America.

Nothing in this addendum, including Section 8’s cross check, independently confirms the oscilloscope's own calibration, trigger timing accuracy, or sample-rate accuracy. Every method compared in this document works from the same captured samples, so a systematic problem upstream of those samples would be invisible to all of them at once. Note also that the measurement endpoint definition is still this project's own provisional reading of the governing standard, contactor state is still only ever known as commanded rather than confirmed, and protective earth continuity is still only inferred indirectly.

10.2 Lessons Learned

The clearest thread connecting this campaign back to the commissioning story in the System Design report is that a system judged trustworthy in advance can still surface things nobody anticipated once it is actually run at scale.

Retaining full raw timestamps on every cycle is what made it possible to reconstruct process pauses and anomalies within campaigns. Running an independent second analysis method against the same data is what turned up both a limitation in a candidate alternative method (the fast-trip window bias) and an unexplained structural pattern (the banded cross validation scatter) that a simple "does it agree with the production number" check would have missed entirely. And directly inspecting the raw samples behind the tightest real margin in the dataset, rather than treating an 11 µs gap as a number to report and move past, is what surfaced the residual floor/quantization finding.

11. Limitations and Recommendations

11.1 Limitations

Some limitations unresolved from this campaign include:

- Contactor physical state readback

- Independent protective earth continuity monitoring
- Confirmation of the measurement endpoint definition against the published UL 2231-2 text
- The external monitoring service's full pause/resume lifecycle was not independently reconfirmed by anything in this document
- Full watchdog, reboot, and true power loss recovery commissioning
- Calibration grade uncertainty terms (oscilloscope timebase accuracy, probe tolerance) that are not available as of writing this report
    - I recommend finding the Calibration Certificate from Esco and noting the specific values in this report after the fact.

A new limitation found includes the collapse side residual floor endpoint refinement mechanism identified in Section 5.4. It is structurally relevant to roughly 4% of the dataset (239 of 5,999 cycles within 500 µs of the boundary), and has not yet fixed or revalidated.

The auto retry mechanism was exercised one time and resolved cleanly, but cannot be used as exhaustive proof of its success. The equipment refresh mechanism's real world activation remains unconfirmed as well.

Other new limitations that are still unresolved include:

- The trimodal shape of the tri -time distribution as seen in Section 5.2
- The banded, stepped structure in the alternative algorithm comparison as seen in Section 8.3
- Whether the moderate correlation between trip time and each waveform's reference amplitude and noise level reflects a genuine property of the device under test or is partly an artifact of the production algorithm's own amplitude derived thresholds
- The residual floor endpoint sensitivity mechanism itself as seen in Section 5.4

11.2 Recommendations and Open Actions

Hardware and physical commissioning work:

1. Add auxiliary contact or voltage feedback to confirm K1/K2/K3 physically reach a commanded state.
2. Add independent, continuously monitored protective earth continuity sensing.
3. Obtain and record oscilloscope timebase and probe calibration data, to complete the uncertainty budget's calibration terms.

Offline Analysis Work:

1. Confirm the measurement endpoint definition against the published UL 2231-2 text

    a. Soon to be resolved as of meeting with Mr. Jeffrey Hemingway, Dr. Rakiul Islam, Mr. Joshua Haines, Mr. Kris Glassford, and myself on 08/28/2026
2. Tighten the collapse side residual floor endpoint refinement.
    a. Require a firmer, quantified margin above the ADC's own quantization step, and/or a sustained run confirmation on the collapse side matching V3's existing onset confirmation principle. Ship this as a new, explicitly validated `AnalysisVersion`, replayed against this campaign's archived waveforms and not as an in-place edit so that future readers are able to view the progression.
3. Investigate the trip time distribution's trimodal structure and the banded cross validation scatter.
4. Define campaign level statistical acceptance criteria.

Instrumentation configuration to reconsider for future campaigns:

1. Implement DC coupling on channel 1.
2. Potentially implement 20 ms/div timescale settings.
    a. I recommend no action required, but it is up to the preference of the future operator and evaluator.

Requires further hardware campaigns to validate:

1. Exercise the persistent deployment pattern for real (never yet used to produce any real campaign), or just retire the feature.
2. Prove full watchdog, reboot, and true power loss recovery.
3. Reconfirm the external monitoring service's full pause/resume lifecycle before the next long unattended run.
4. Continue accumulating real world evidence for auto retry and equipment refresh correctness.

Appendix A. Full Data Tables

A.1 Campaign Summary

Run ID	Target	Completed	Pass	Fail	No Trip
`200_v3_real_20260813T131932Z`	200	200	197	3	0
`5800_v3_try2_20260813T195018Z`	5,800	483	471	12	0
`5317_v3_real_20260817T143315Z`	5,317	5,317	5,215	101	1
Combined		6,000	5,883	116	1

A.2 Time Trip Distribution

Statistic	Value
Mean	16.77 ms
Standard Deviation	4.07 ms
Minimum	7.69 ms
5th percentile	11.55 ms
25th percentile	12.89 ms
Median	16.53 ms
75th percentile	19.16 ms
95th percentile	24.25 ms
99th percentile	25.15 ms
Maximum	25.56 ms

A.3 Boundary Cycles

Cycle	Trip time	Margin
Slowest PASS	24.9605 ms	9.5 µs below the 24.97 ms pass limit
Fastest FAIL	24.9715 ms	1.5 µs above the pass limit
Slowest FAIL	25.5580 ms	74.44 ms below the 100 ms no-trip limit
NO_TRIP	--	No collapse observed within the record

A.4 Drift Regression

Scope	n	Slope	r	p
5,317 cycle run only, PASS	5215	-0.0225 µs/cycle	-0.0088	0.527
5,317 cycle run only, PASS + FAIL	5316	-0.0303 µs/cycle	-0.0114	0.406
Full combined timeline, PASS	5883	-0.0118 µs/cycle	-0.0052	0.690
Full combined timeline, PASS + FAIL	5999	-0.0201 µs/cycle	-0.0086	0.508

A.5 Correlations

Pair	r	p	R^2
Trip time vs. reference amplitude	0.4447	2.27×10^{289}	19.8%
Trip time vs. noise level	-0.5873	$< 10^{-300}$	34.5%
Reference amplitude vs. cycle index	0.134	≈ 0	1.8%

A.6 Cross Validation Summary

Method	Mean offset from production algorithm	Median offset	Std	PASS/FAIL flips after offset correction (of 5,999)
Threshold based (Methods A/B)	+2.407 ms	+2.393 ms	1.457 ms	27
Curve-fit (Method C)	-1.897 ms	-1.988 ms	2.025 ms	62

A.7 Verdict Proportions with 95% Wilson Confidence Intervals

Verdict	k	n	Proportion	95% CI
PASS	5883	6000	98.050%	(97.668%, 98.37%)
FAIL	116	6000	1.933%	(1.614%, 2.314%)
NO_TRIP	1	6000	0.017%	(0.003%, 0.094%)

A.8 Per Run Breakdown

Run	n	Pass rate	95% Wilson CI	Mean	Std	Min	Median	Max
`200_v3_real`	200	98.500%	(95.683%, 99.489%)	16.60 ms	3.86 ms	7.89 ms	16.50 ms	25.56 ms
`5800_v3_try2`	483	97.516%	(95.708%, 98.573%)	16.84 ms	3.98 ms	7.98 ms	16.59 ms	25.41 ms
`5317_v3_real`	5,317	98.082%	(97.677%, 98.417%)	16.77 ms	4.08 ms	7.69 ms	16.53 ms	25.44 ms

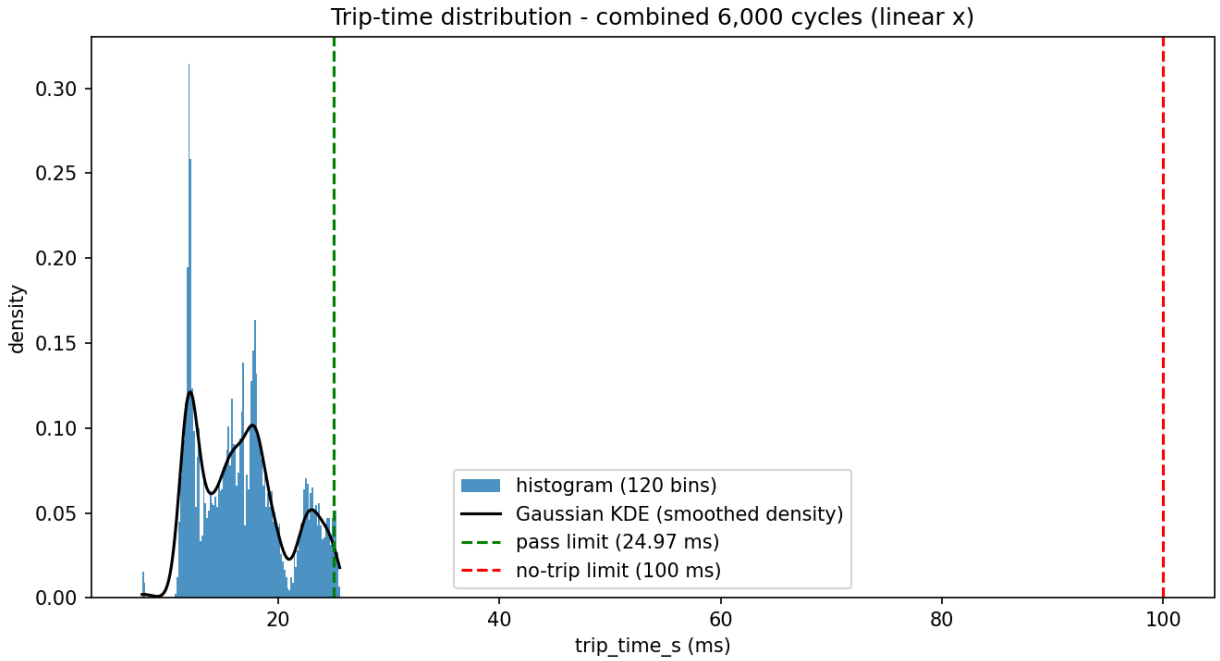
A.9 Near Boundary Cycle Counts

Distance from Pass Limit	Cycle Count
Within 50 µs	23
Within 100 µs	48
Within 500 µs	239
Within 1 ms	361

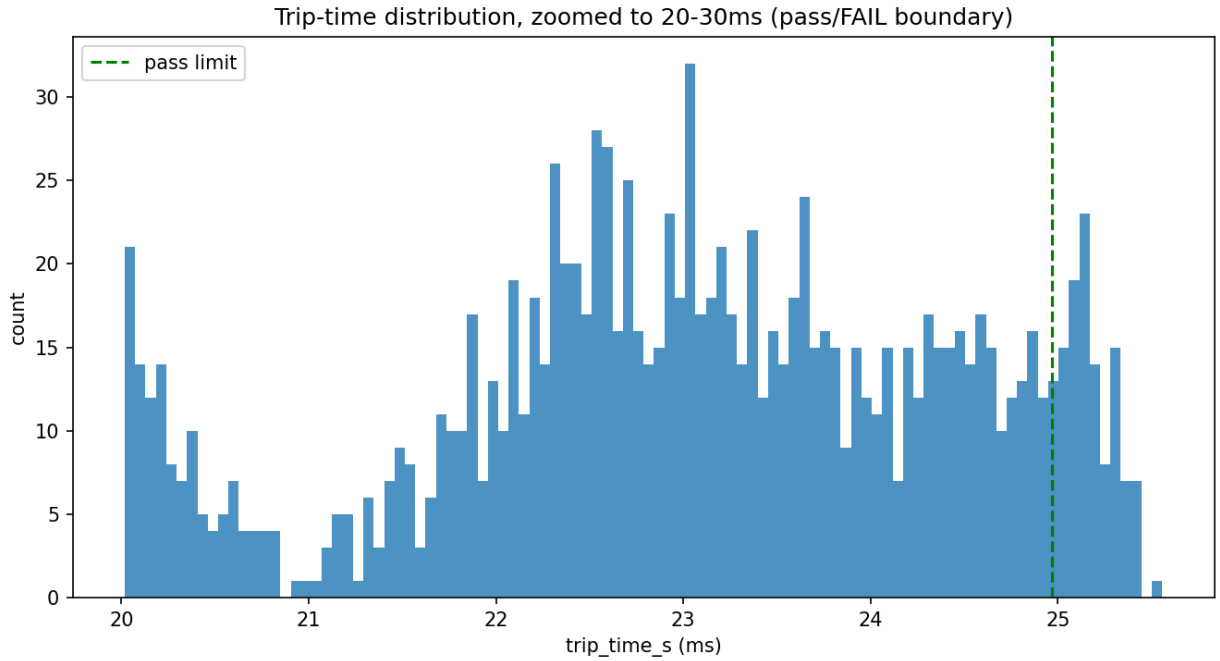
Appendix B. Plots

B.1 Distribution and Verdict Plots

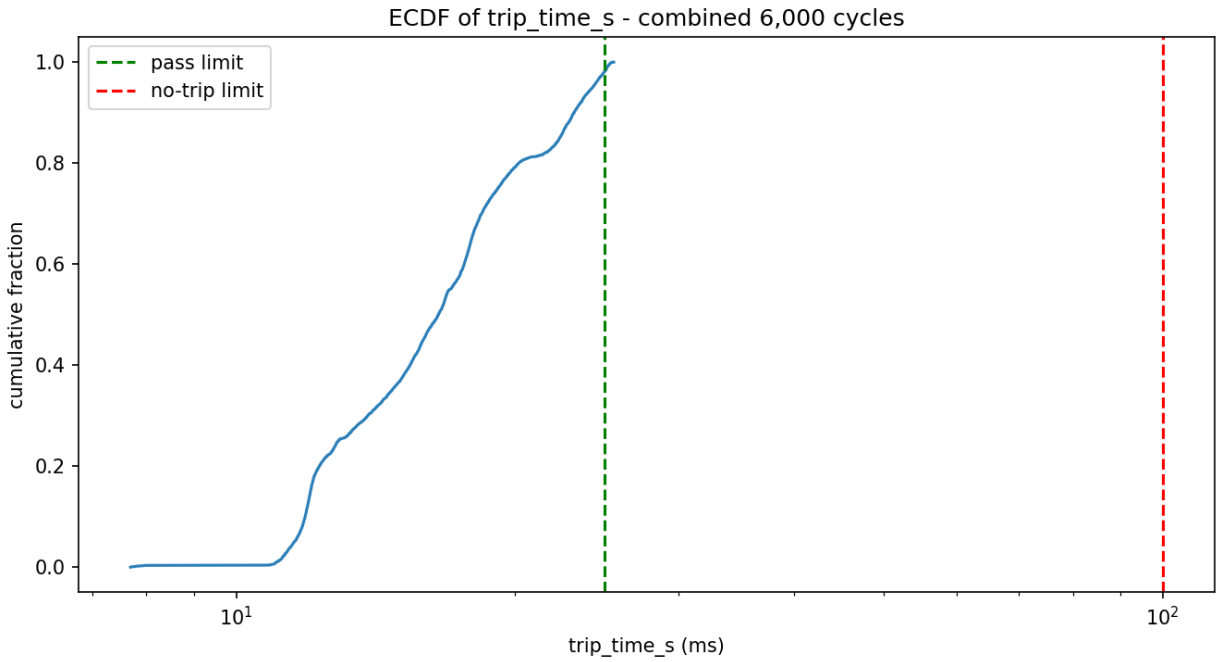
B.1.1 Trip Time Histogram



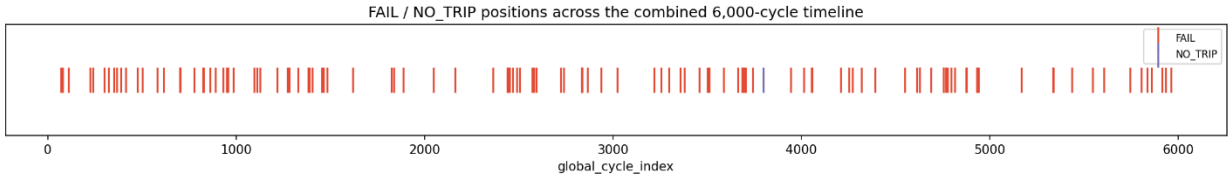
B.1.2 Trip Time Histogram, Zoomed to Pass/Fail Boundary



B.1.3 Trip Time Empirical Cumulative Distribution

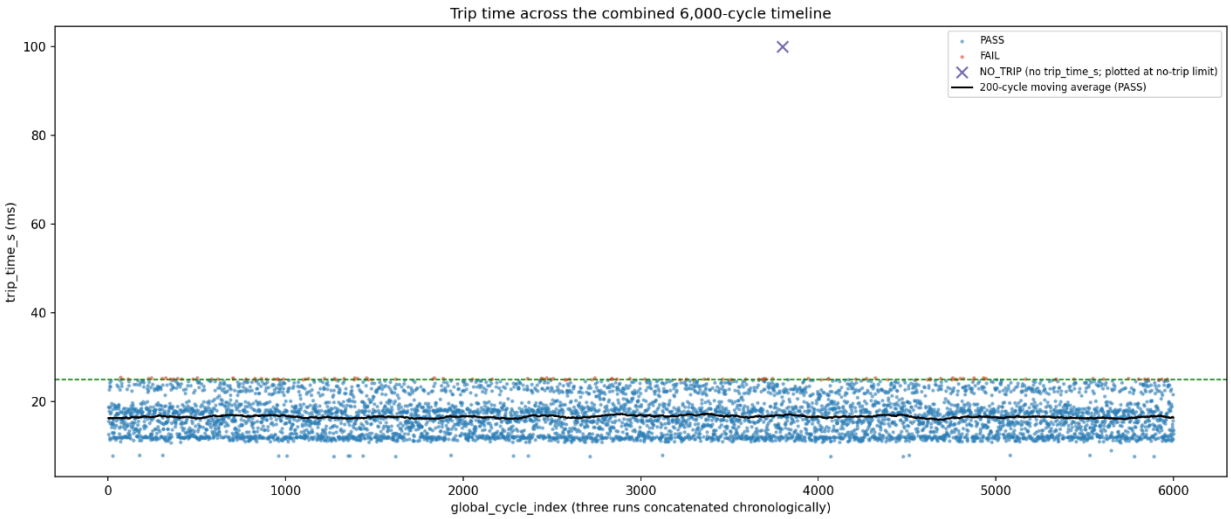


B.1.4 Verdict Timeline Strip

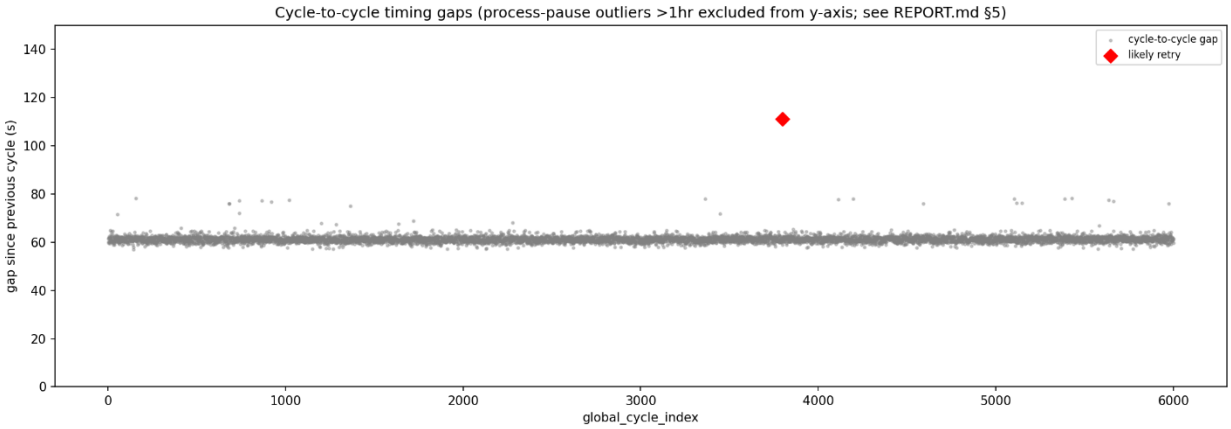


B.2 Timeline Plots

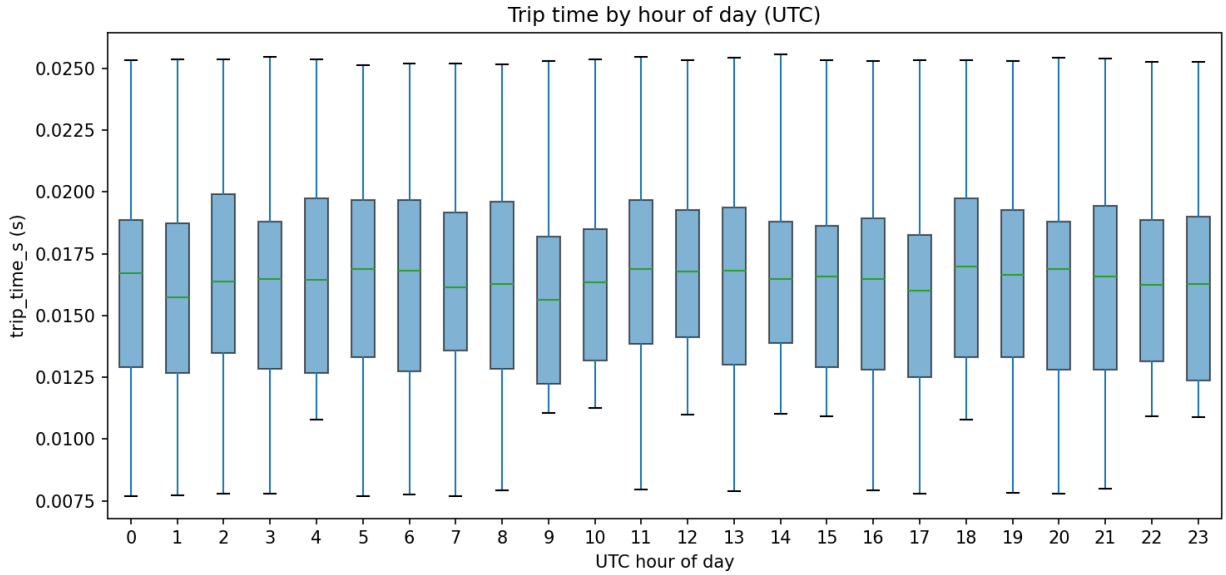
B.2.1 Trip Time vs. Combined Cycle Index



B.2.2 Cycle to Cycle Timing Gap Timeline

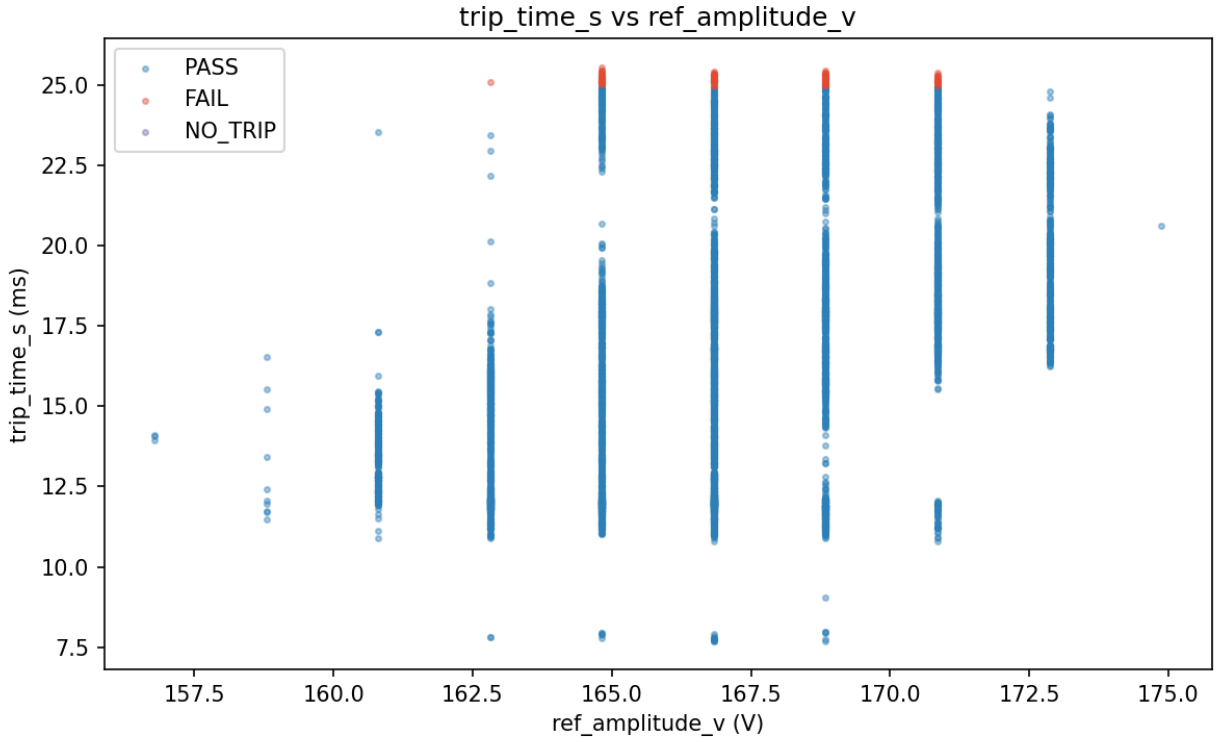


B.2.3 Trip Time and Pass Rate by Hour of Day

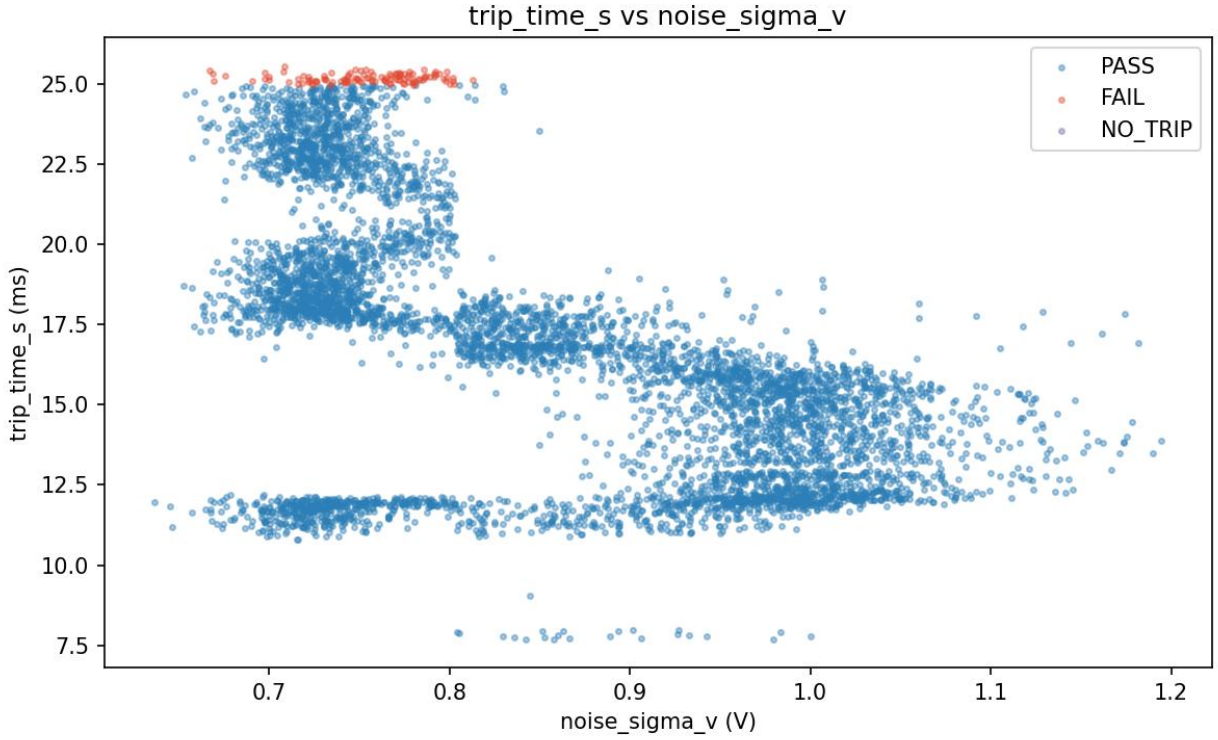


B.3 Correlation Plots

B.3.1 Trip Time vs. Waveform Reference Amplitude

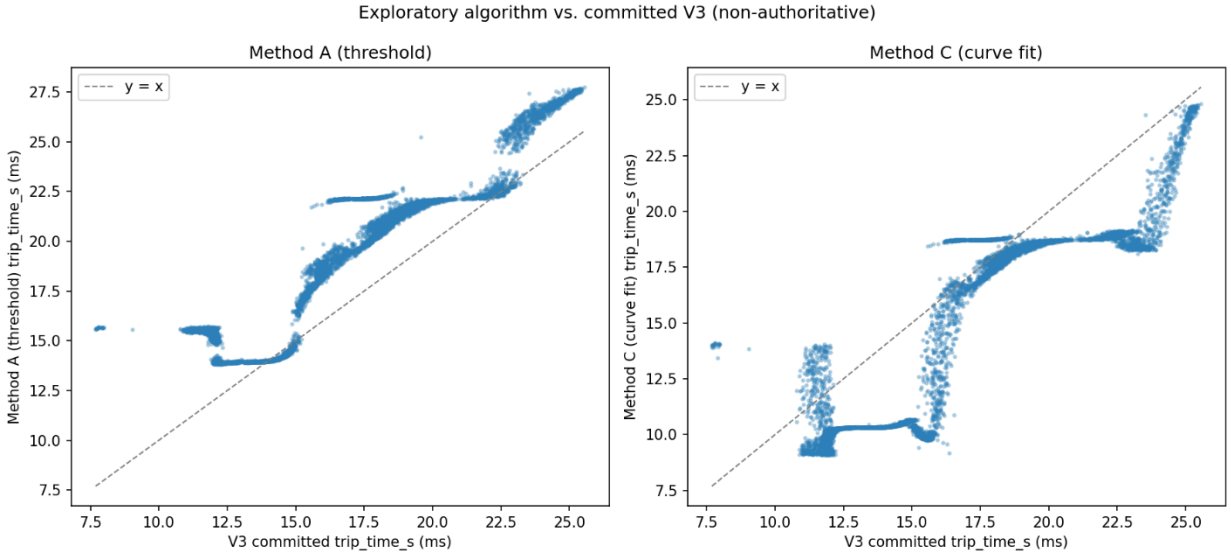


B.3.2 Trip Time vs. Waveform Noise Level



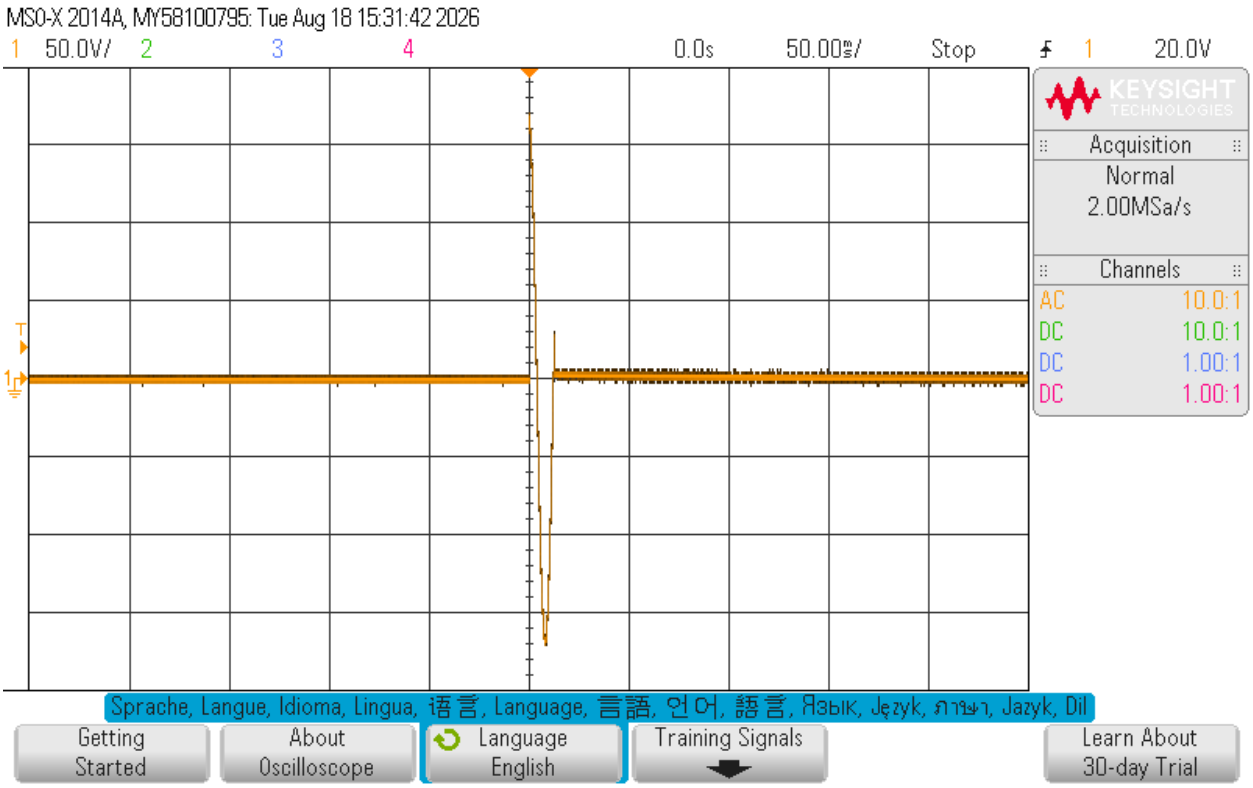
B.4 Cross Validation Plots

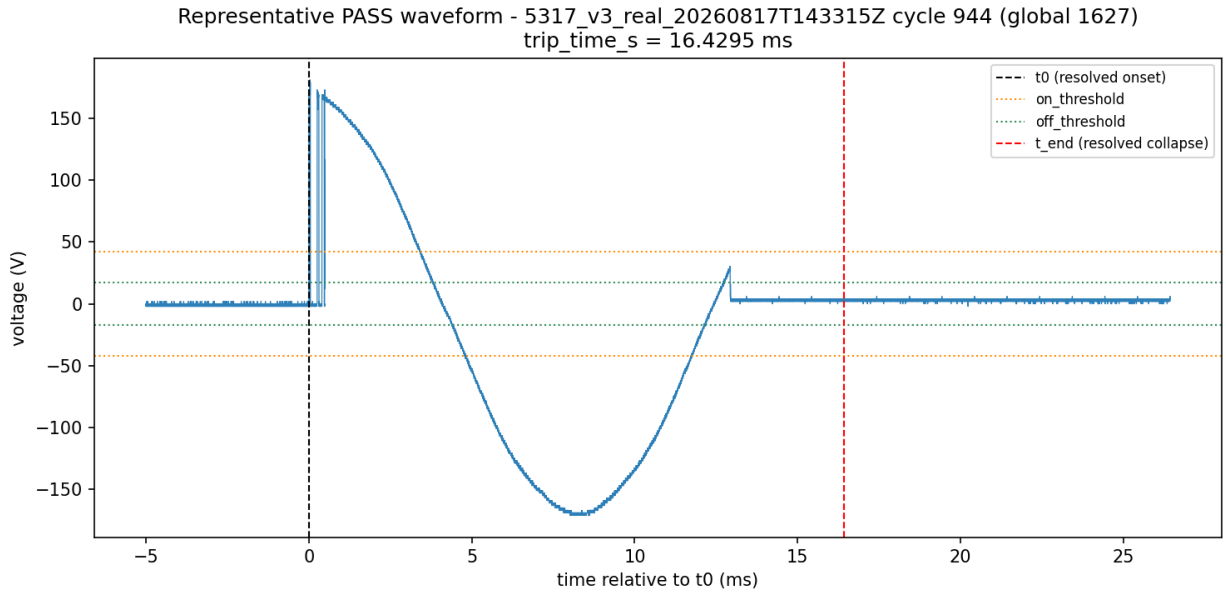
B.4.1 Alternative Method Trip Time vs. Production Algorithm Trip Time



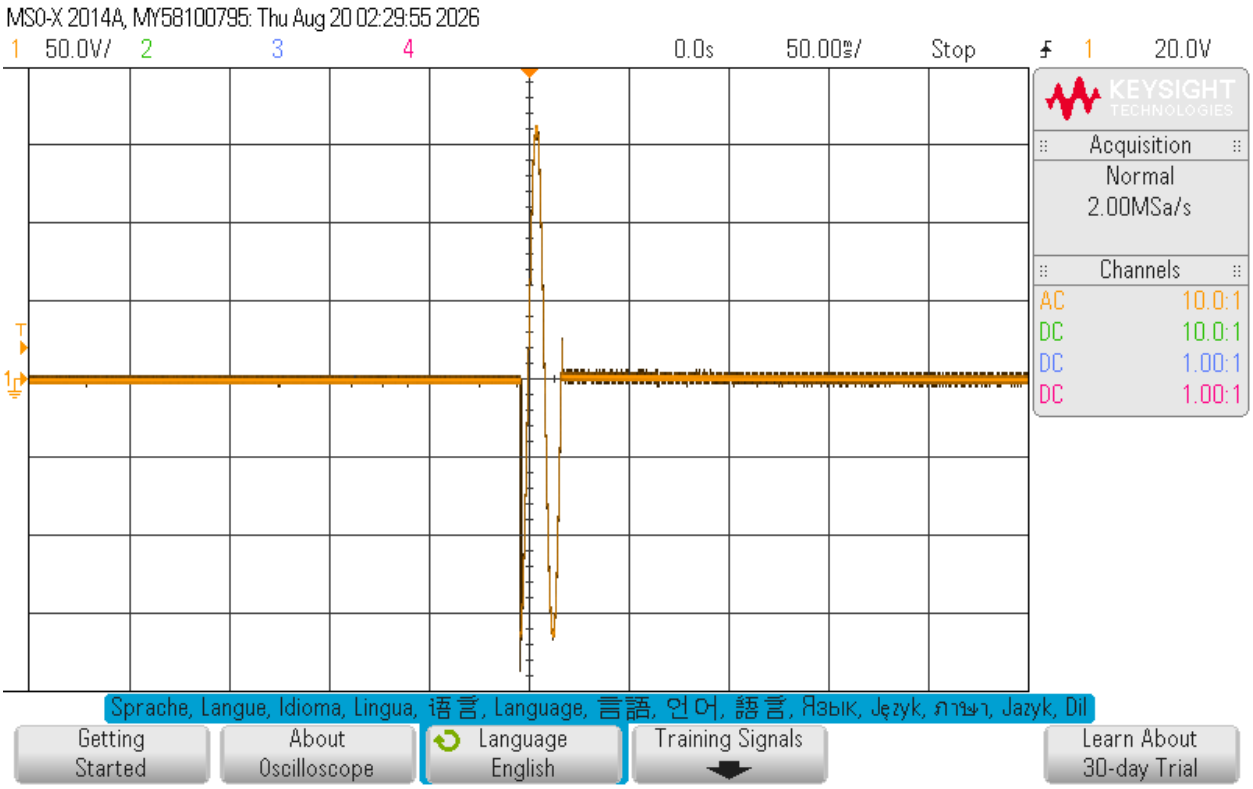
B.5 Representative Waveforms

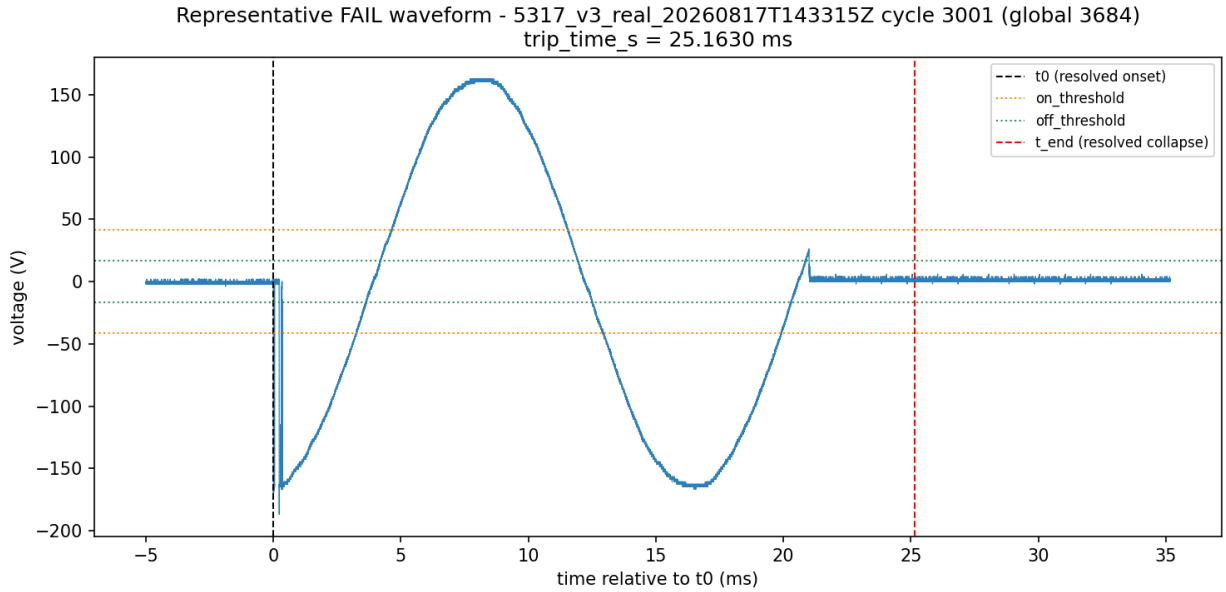
B.5.1 Representative Pass Waveform (Cycle 944, Global 1,627)





B.5.2 Representative FAIL Waveform (Cycle 3,001, Global 3,684)





B.5.3 Representative NO TRIP Waveform (Cycle 3,115, Global 3,789)